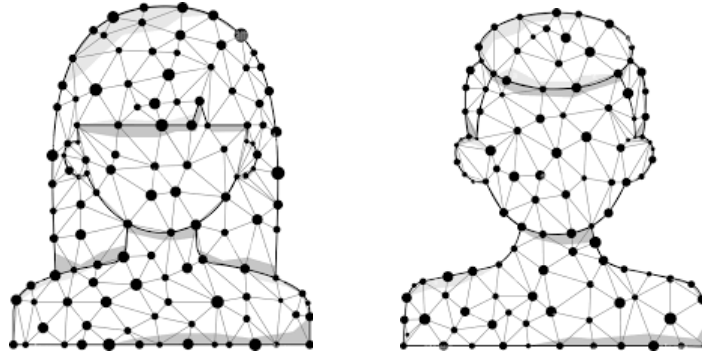




Tecniche e applicazioni biometriche

Fabio Scotti
fabio.scotti@unimi.it

Università degli Studi di Milano



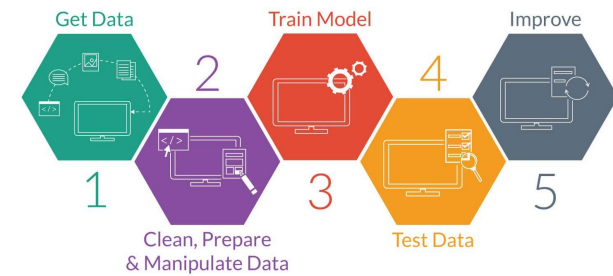
Progettazione di modelli di Machine Learning per applicazioni biometriche

Fabio Scotti

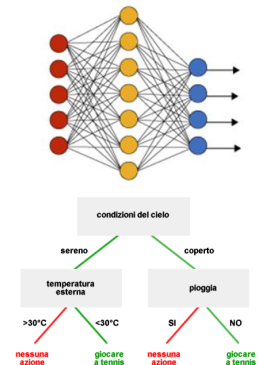
fabio.scotti@unimi.it

Università degli Studi di Milano

Outline



1. Progettazione di modelli di Machine Learning
 1. Flusso di lavoro
 2. Feature dei modelli da considerare per al scelta
 3. Consigli riassuntivi
2. Introduzione alla Computational Intelligence (CI)
3. Reti neurali feed-forward
4. Curse of Dimensionality
5. Modelli tradizionali di AI (non CI)
 1. Classificatore ai primi vicini kNN
 2. Albero decisionale
6. Caratteristiche dei modelli di Deep Learning



A close-up photograph of a person's hand holding a black pen, drawing on a set of architectural blueprints. A yellow ruler is placed horizontally across the lower part of the frame. The person is wearing a silver-toned metal watch with a blue dial. The background is slightly blurred, showing more of the drawing process.

Design of machine learning models

Suggerimenti per migliorare la progettazione

Premio Nobel della fisica

John Hopfield e Geoffrey Hinton

Per le scoperte e invenzioni fondamentali che consentono l'apprendimento automatico con **reti neurali artificiali**

<https://www.nobelprize.org/prizes/physics/2024/press-release/>

John Hopfield

- ha inventato una rete che utilizza un metodo per salvare e ricreare schemi.
- Possiamo immaginare i nodi come pixel.
- La rete di Hopfield sfrutta la fisica che descrive le caratteristiche di un materiale in base al suo spin atomico, una proprietà che rende ogni atomo un piccolo magnete. La rete nel suo complesso viene descritta in modo equivalente all'energia presente nel sistema di spin trovato in fisica, e viene addestrata trovando valori per le connessioni tra i nodi affinché le immagini salvate abbiano bassa energia.
- Quando alla rete di Hopfield viene fornita un'immagine distorta o incompleta, essa lavora metodicamente attraverso i nodi e aggiorna i loro valori in modo che l'energia della rete diminuisca. La rete, quindi, lavora passo dopo passo per trovare l'immagine salvata che è più simile a quella imperfetta che è stata fornita.

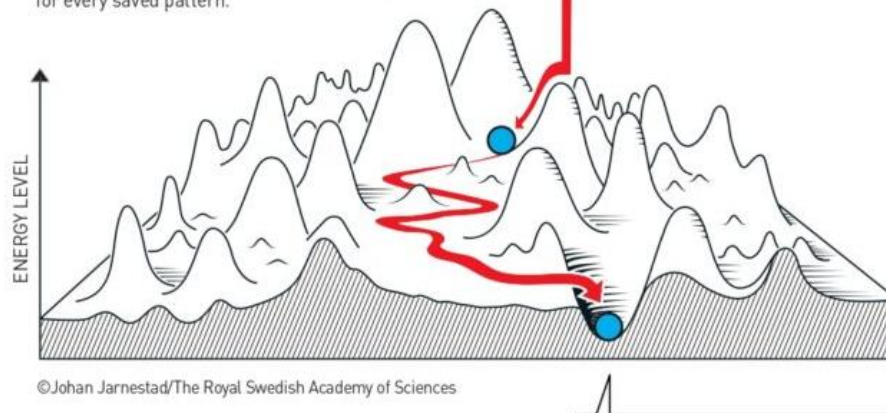
Tuttavia per le equazioni che legano fra loro i neuroni vi sono dei limiti alla capacità rappresentativa (numero di stati stabili/immagini di riferimento)

Uno stato stabile per ogni pattern da memorizzare.

Una immagine in input, anche se rumorosa, iterando «cade» nel corrispondente stato stabile

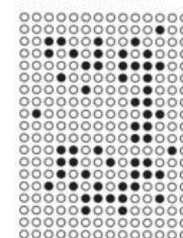
Memories are stored in a landscape

John Hopfield's associative memory stores information in a manner similar to shaping a landscape. When the network is trained, it creates a valley in a virtual energy landscape for every saved pattern.

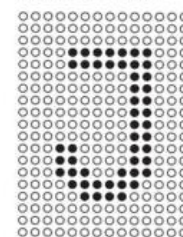


- 1 When the trained network is fed with a distorted or incomplete pattern, it can be likened to dropping a ball down a slope in this landscape.
- 2 The ball rolls until it reaches a place where it is surrounded by uphill. In the same way, the network makes its way towards lower energy and finds the closest saved pattern.

INPUT PATTERN



SAVED PATTERN



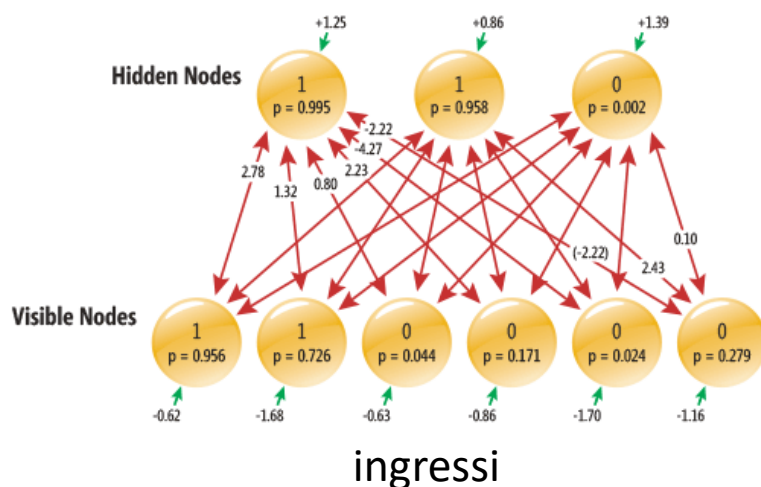
Premio Nobel della fisica

John Hopfield e Geoffrey Hinton

- Per le scoperte e invenzioni fondamentali che consentono l'apprendimento automatico con **reti neurali artificiali**
<https://www.nobelprize.org/prizes/physics/2024/press-release/>

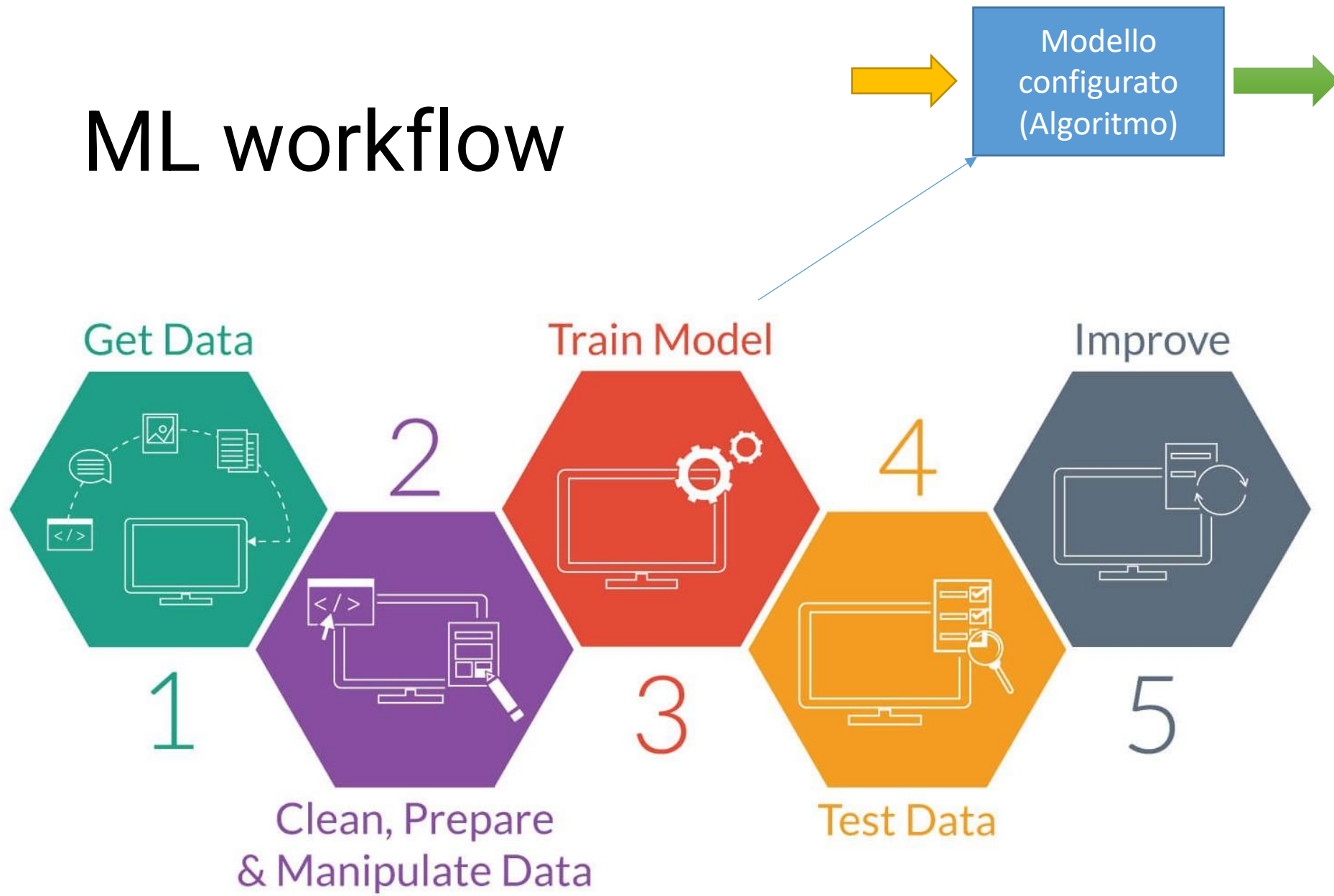
Geoffrey Hinton

- ha utilizzato la rete di Hopfield come base per una nuova rete che impiega un metodo diverso: **la macchina di Boltzmann, un tipo di rete neurale ricorrente stocastica**,
- Questa rete può imparare a riconoscere elementi caratteristici in un determinato tipo di dati.
- Hinton ha usato strumenti provenienti dalla fisica statistica, la scienza dei sistemi costituiti da molti componenti simili.
- La macchina viene addestrata fornendole esempi che sono molto probabili quando la macchina è in funzione. **La macchina di Boltzmann può essere utilizzata per classificare immagini o creare nuovi esempi del tipo di schema su cui è stata addestrata.** Hinton ha costruito su questo lavoro, contribuendo a dare il via allo sviluppo esplosivo attuale del machine learning.



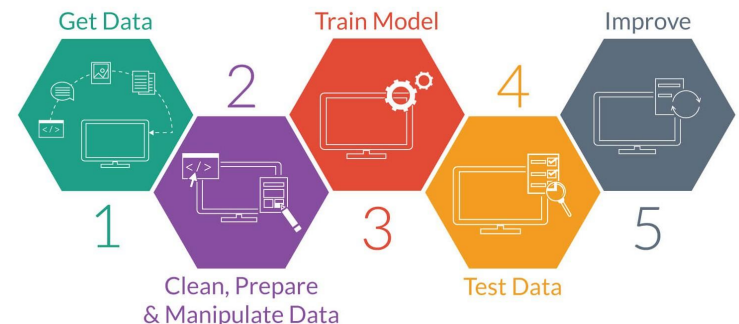
L'introduzione dello **strato nascosto** nelle macchine di Boltzmann **ha aumentato la capacità di rappresentazione** ed elaborazione della rete, segnando una transizione verso strutture più potenti che sono alla base del **deep learning moderno**.

ML workflow

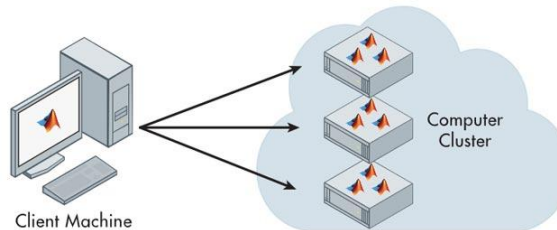
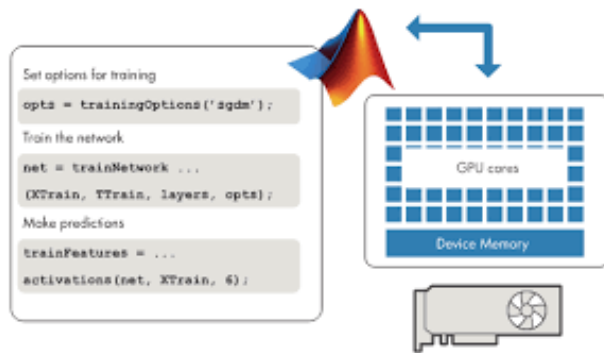
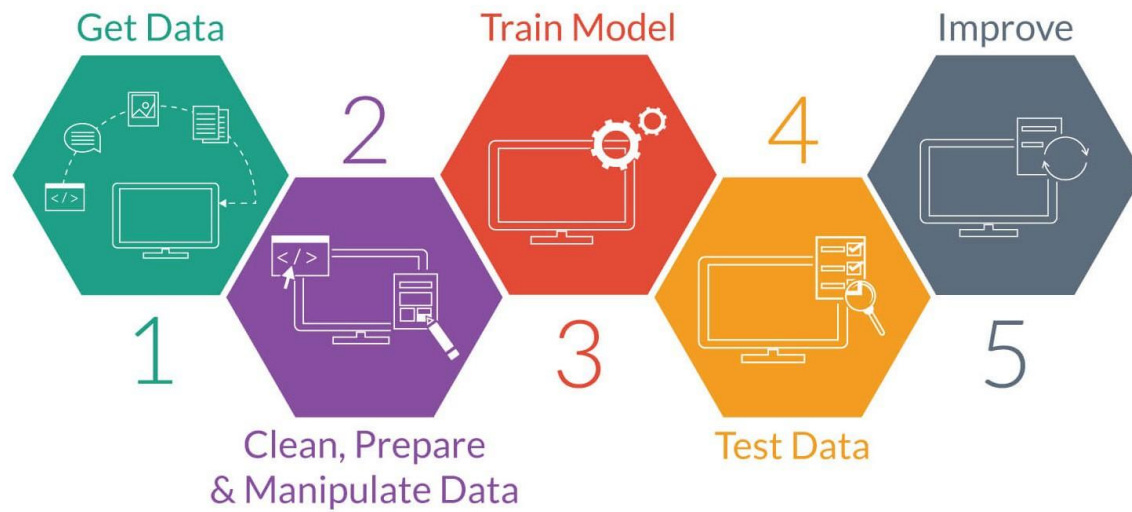


ML in Practice

- Comprensione del dominio, conoscenze pregresse e obiettivi
- Integrazione, selezione, pulizia, pre-elaborazione dei dati, ecc.
- Modelli di apprendimento
- Interpretazione dei risultati
- Consolidamento e distribuzione delle conoscenze scoperte
- Ciclo



Ci sono ambienti completi per tutti i passi 1-5



La tecnica di ML giusta

- La selezione della giusta tecnica di ML richiede una conoscenza approfondita sia del problema che delle tecniche in considerazione.
- I problemi reali possono richiedere una combinazione di tecniche (IA e/o non-IA) per una soluzione ottimale (approcci ibridi).

Selezione della tecnica di ML

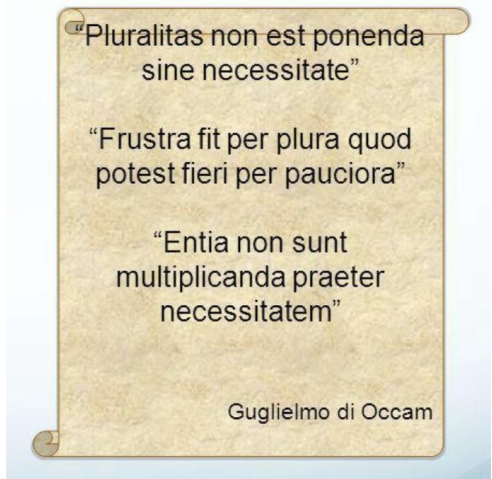
- Studia i requisiti del compito
- Valuta quale tecnica soddisfa quei requisiti in modo più completo
- Classifica le possibili soluzioni rispetto alla:
 - Applicabilità,
 - Costo,
 - Prestazioni,
 - ...

Features del modello da considerare

- **Accuracy**
- Explainability
- **Response speed**
- **Scalability**
- Compactness
- Flexibility
- **Embedability**
- Ease of use
- Learning curve
- Tolerance for complexity
- **Tolerance for noise in data**
- Tolerance for sparse data
- Independence from experts
- Development speed
- Computing ease/difficulty

Selezione della tecnica “migliore”

- Un dato problema può essere risolto in molti modi diversi
- Anche se due tecniche producono soluzioni di qualità simile, abbinare la giusta tecnica a un problema può **risparmiare tempo e risorse**
- Caratteristiche di una tecnica ottimale:
 - contiene tutte le informazioni richieste
 - soddisfa tutti gli altri criteri necessari
 - utilizza tutte le conoscenze disponibili (utili)

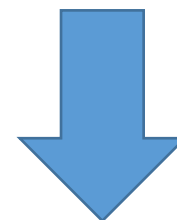
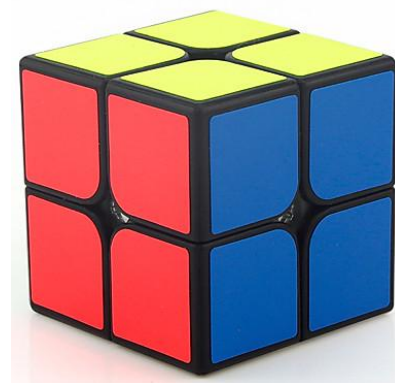


«Si fa inutilmente con molte cose
ciò che si può fare con poche cose»



Effetti del rasoio di Occam

1. Per non perdere tempo
 - Mentale
 - Di elaborazione (modelli impressionanti!)
2. ... usare prima modelli «classici» dei modelli neurali. Es:
 - KNN (classificatore ai primi vicini)
 - DTREE (albero decisionale)
3. Se servono modelli neurali iniziare con pochi neuroni e non deep
4. Aumentare i numero di strati se serve
5. Passare ai modelli deep (a molti strati e molti neuroni) se i passi precedenti dimostrano che è necessario.



Usare il ML in biometria



Hugging Face

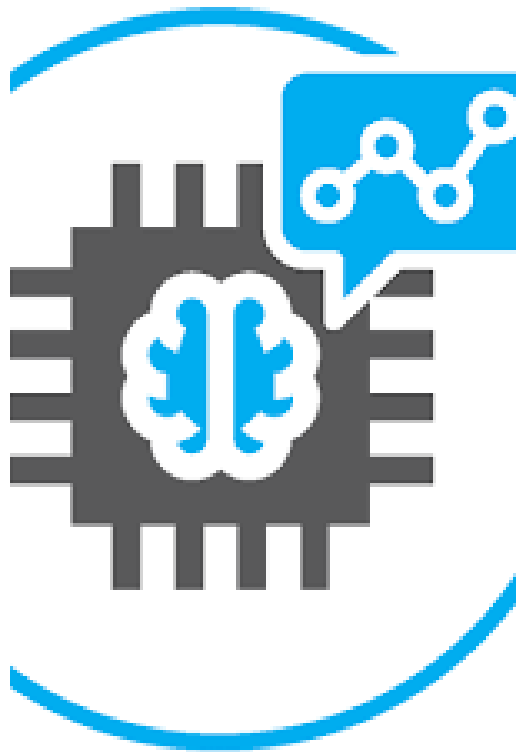


GitHub

Due possibilità

1. Usare modelli/reti **già allenati** per uno scopo specifico (face detection,...)
2. Procedere ad un **allenamento**
 1. **Completo**
 - Specialmente per le reti deep servono
 - Moltissimi dati
 - Moltissima potenza computazionale per il training
 - **Parziale** (possibile tipicamente con modelli deep)
 - Meno dati e risorse ma occorre seguire procedure esattamente per la biometria

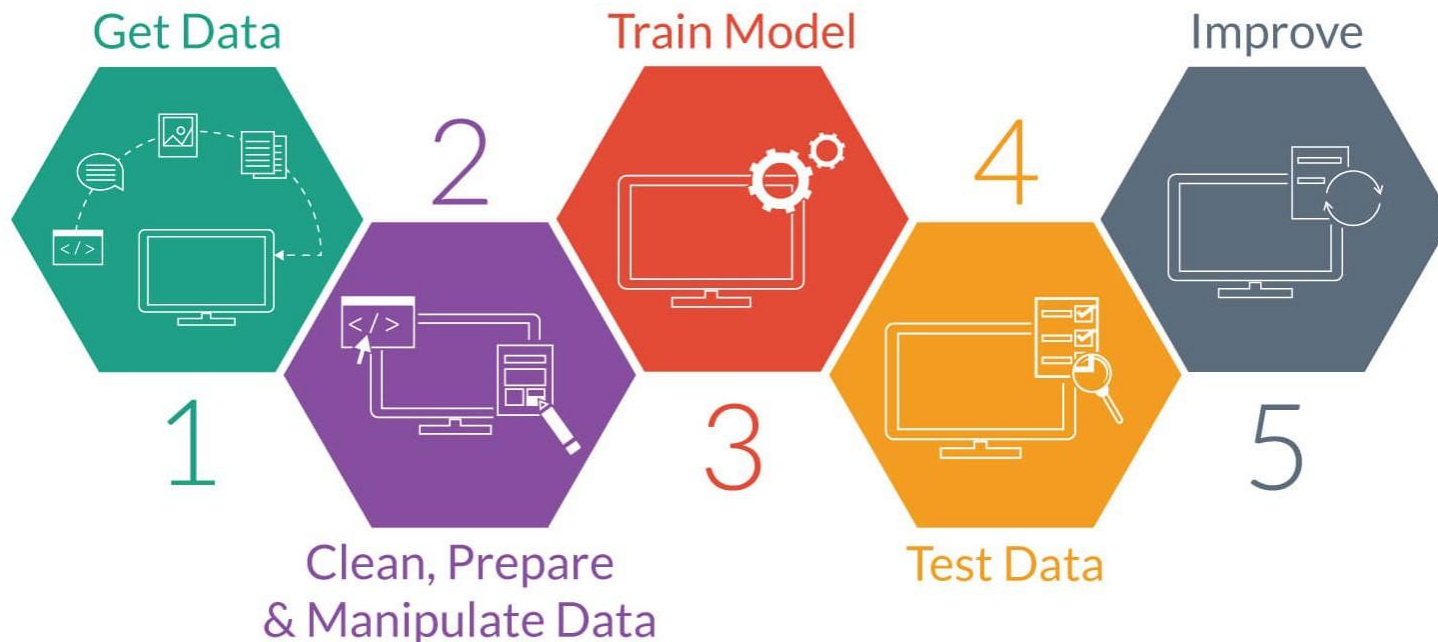
Usare il ML in biometria: 4 punti importanti



1. Seguire un corretto flusso di lavoro per non perdere tempo o produrre risultati scudenti
2. Usare partizioni dei dati corretti per applicazioni biometriche come la L(NP)O e non solo quelle che fornisce la libreria/tool!
3. Preferire soluzioni semplici e partire dal semplice
4. Usare tecniche avanzate come il transfer learning

1) Procedere ad progettazione → Seguire il workflow

Non si saltano passaggi o si fa un solo giro....
Molto meglio pianificare e investire tempo in
queste attività



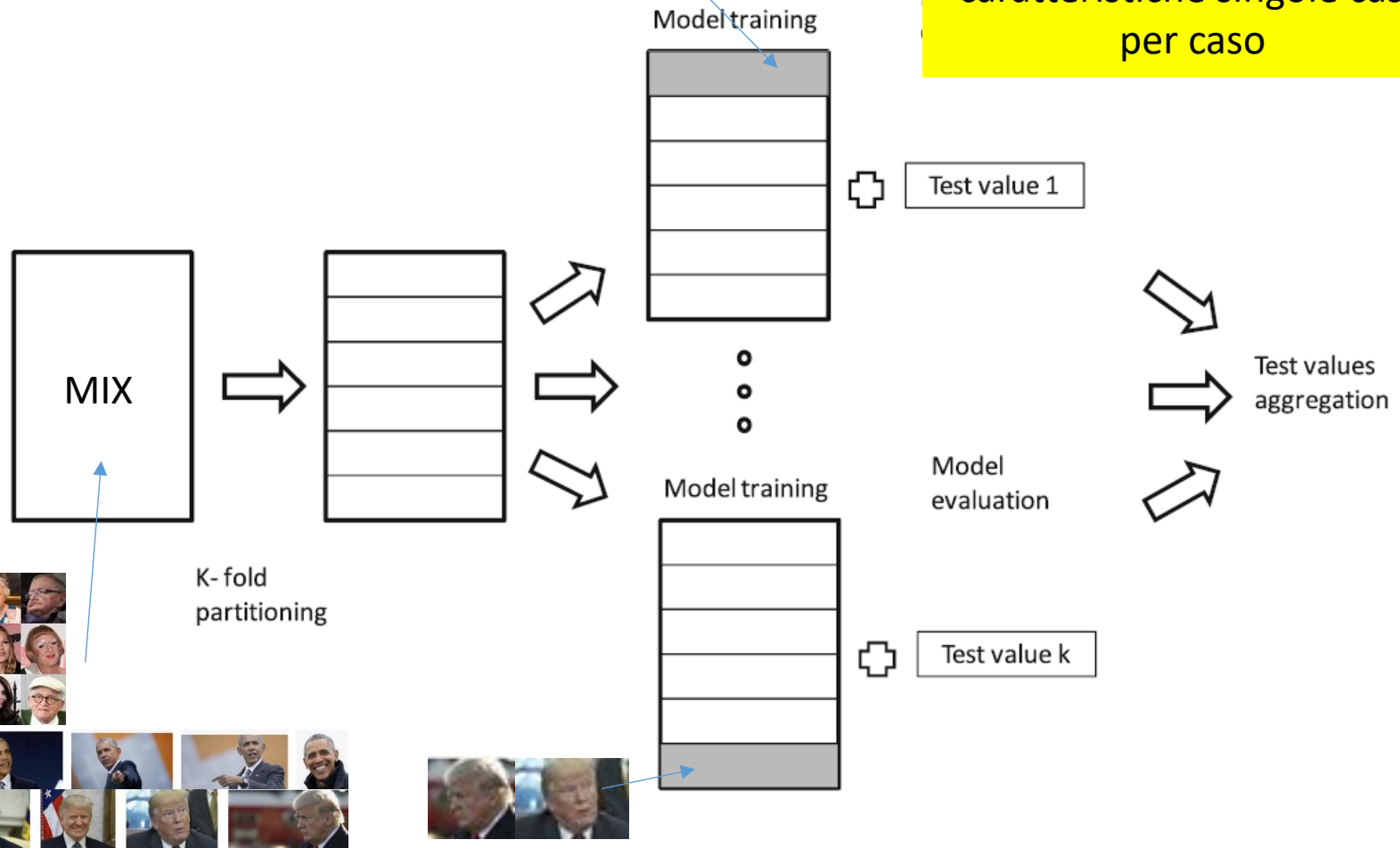
2) Procedere ad progettazione

Usare L(NP)O

(se possibile)



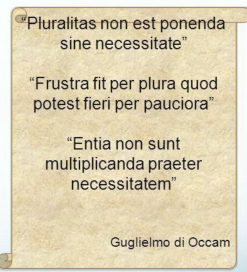
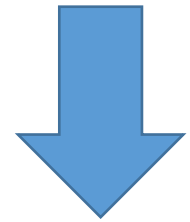
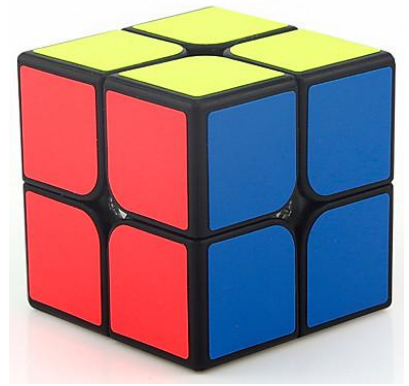
Forziamo il modello di ML
a imparare ad usare le
caratteristiche
discriminatorie generali
(biometriche)
fra gli individui e non
caratteristiche singole caso
per caso



3) Procedura di progettazione

Preferire soluzioni semplici

- Vedi rasoio di OCCAM

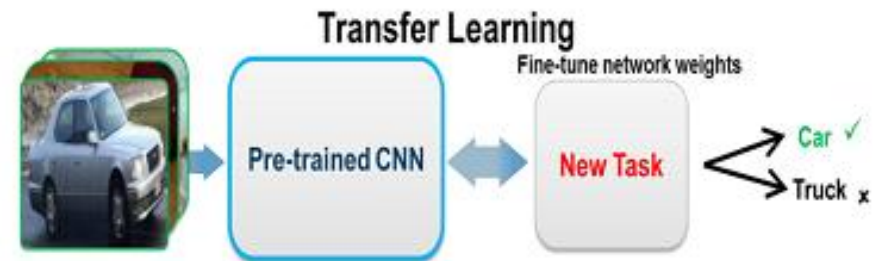


«Si fa inutilmente con molte cose
ciò che si può fare con poche cose»

4) Procedura di progettazione

Considerare tecniche di Transfer Learning

Fine tuning di Pre-trained Convolutional Neural Network



Training data	1000s to millions of labeled images
Computation	Compute intensive
Training Time	Days to Weeks for real problems
Model accuracy	High (can over fit to small datasets)

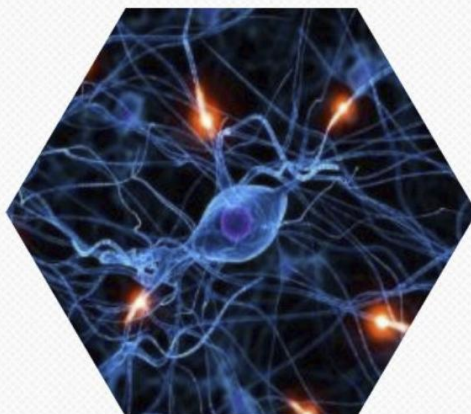
Training data	100s to 1000s of labeled images (small)
Computation	Moderate computation
Training Time	Seconds to minutes
Model accuracy	Good, depends on the pre-trained CNN model

I 3 principali drivers in AI

More Data



Better Models and Algorithms



Powerful GPU Accelerators



Specialmente per il Deep Learning

Ma il 4° driver è il più importante

- Il cervello del designer!
 - A priori knowledge
 - Data selection
 - Data Filtering and Enhancing
 - Model selection
 - Learning technique choice
 - Experiment Design
 - Avoid brute force
 - Hybrid systems
 - Dividi et impera





Principali modelli induttivi per il ML

Applicazioni biometriche (non solo reti neurali....)

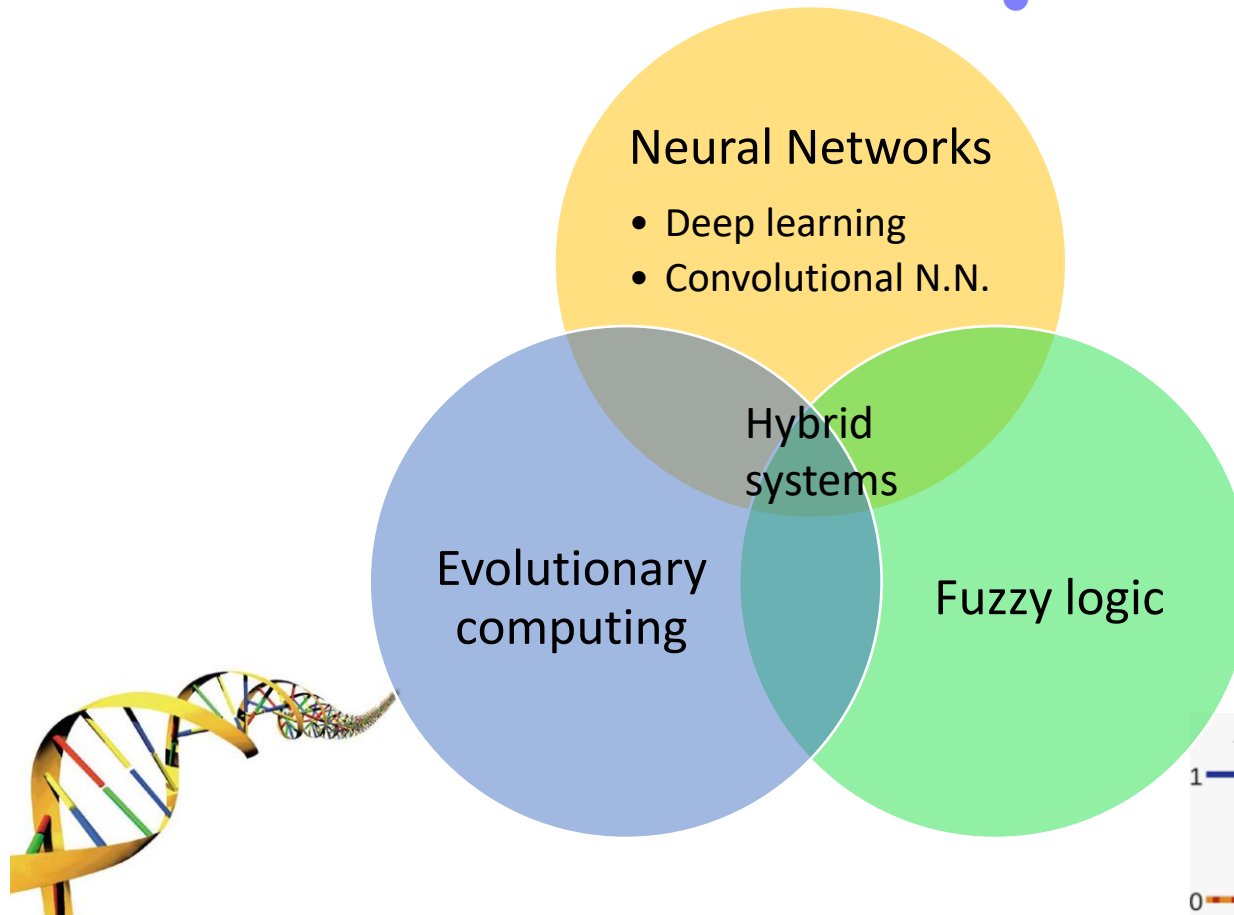
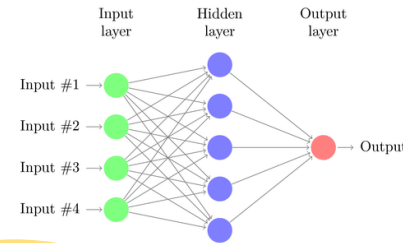
Computational intelligence

- Secondo la *IEEE computational intelligence society* si ha la seguente definizione

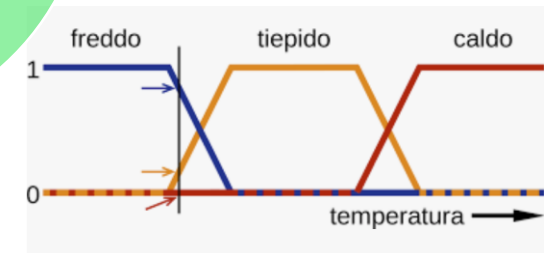
“theory, design, application, and development of
biologically and linguistically motivated
computational paradigms emphasizing
neural networks, connectionist systems (*),
genetic algorithms,
evolutionary programming,
fuzzy systems,
and **hybrid intelligent systems**
in which these paradigms are contained”

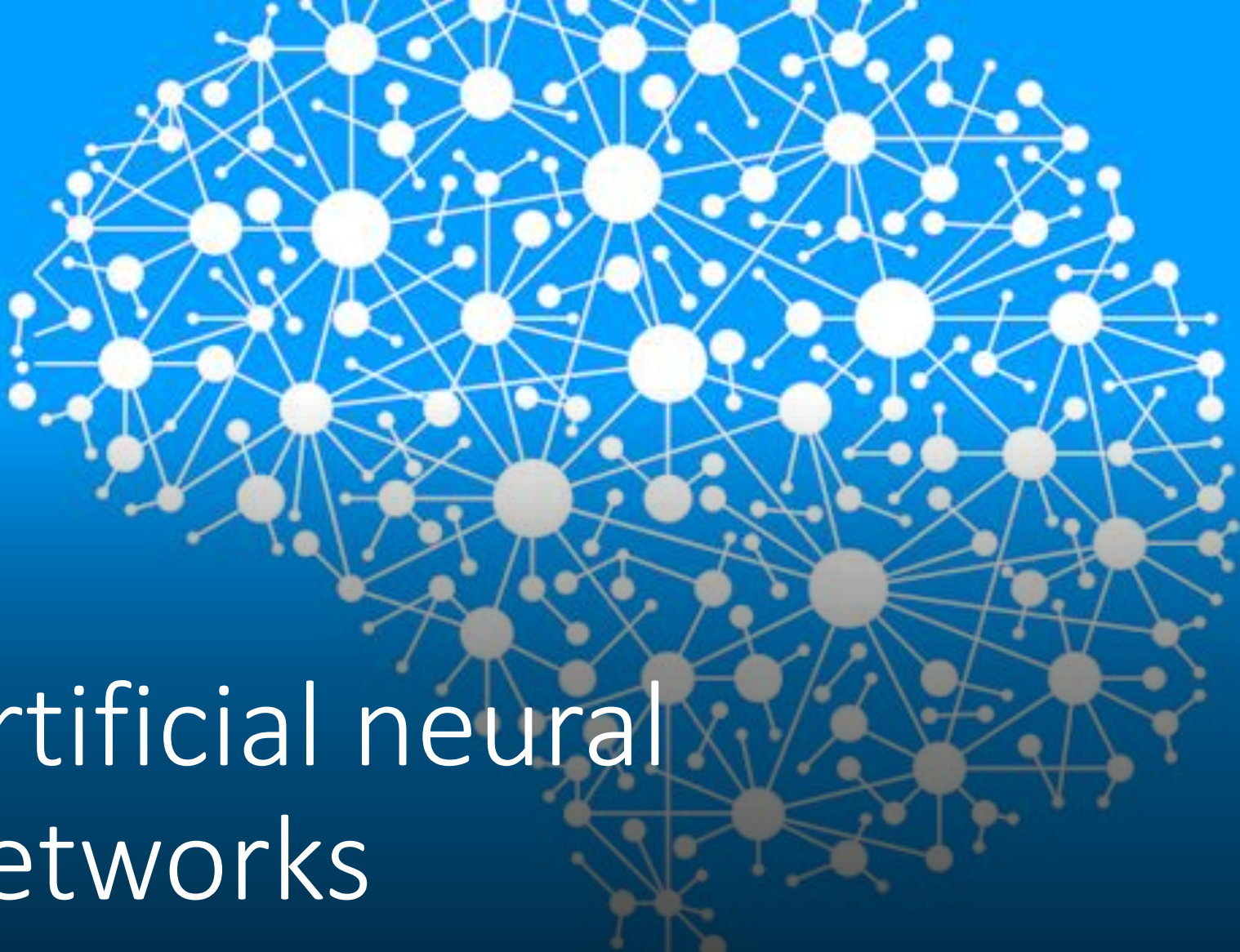
(*) represents mental or behavioral phenomena as emergent processes of interconnected networks of simple units.

Computational Intelligence (CI) main techniques



Questi metodi vengono approfonditi nel corso [Artificial intelligence](#) e [Intelligent systems for industry, supply chain and environment](#)

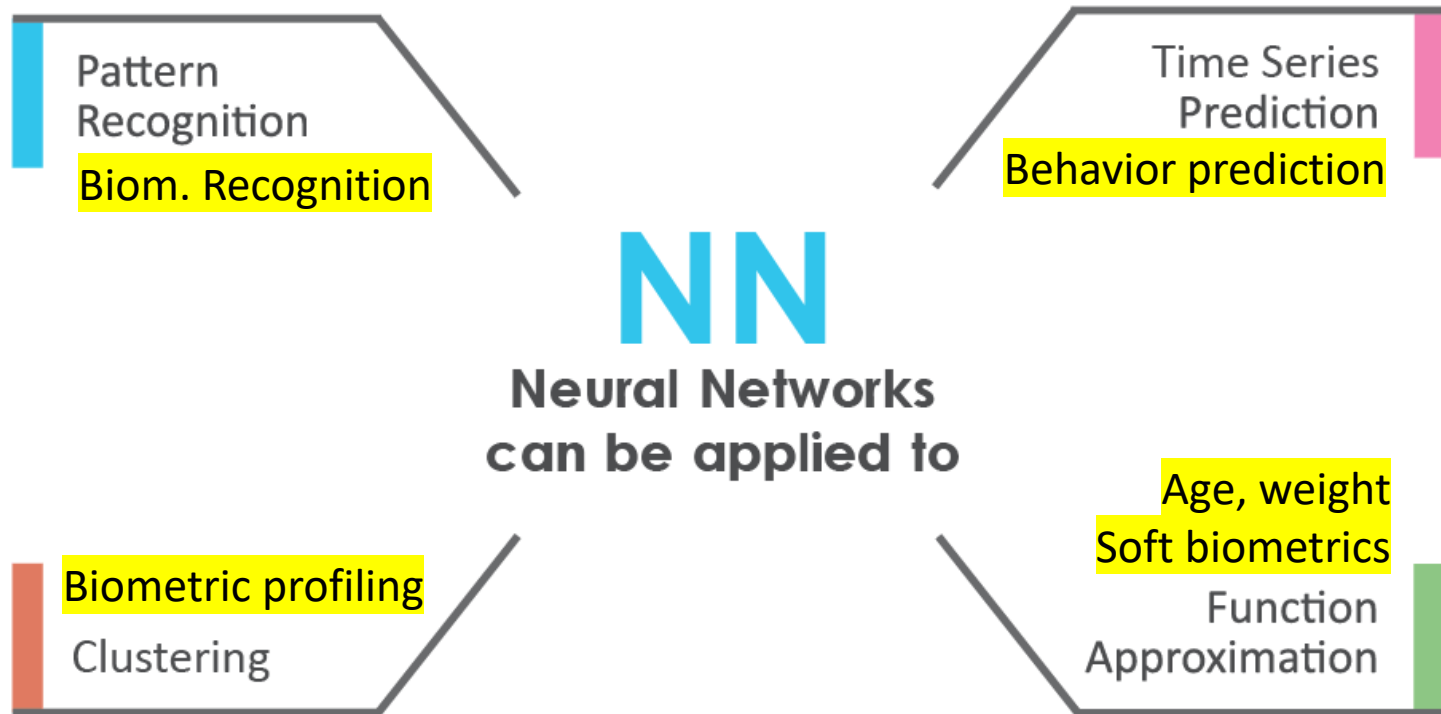




Artificial neural networks

Uno dei modelli di ML più applicato in biometria

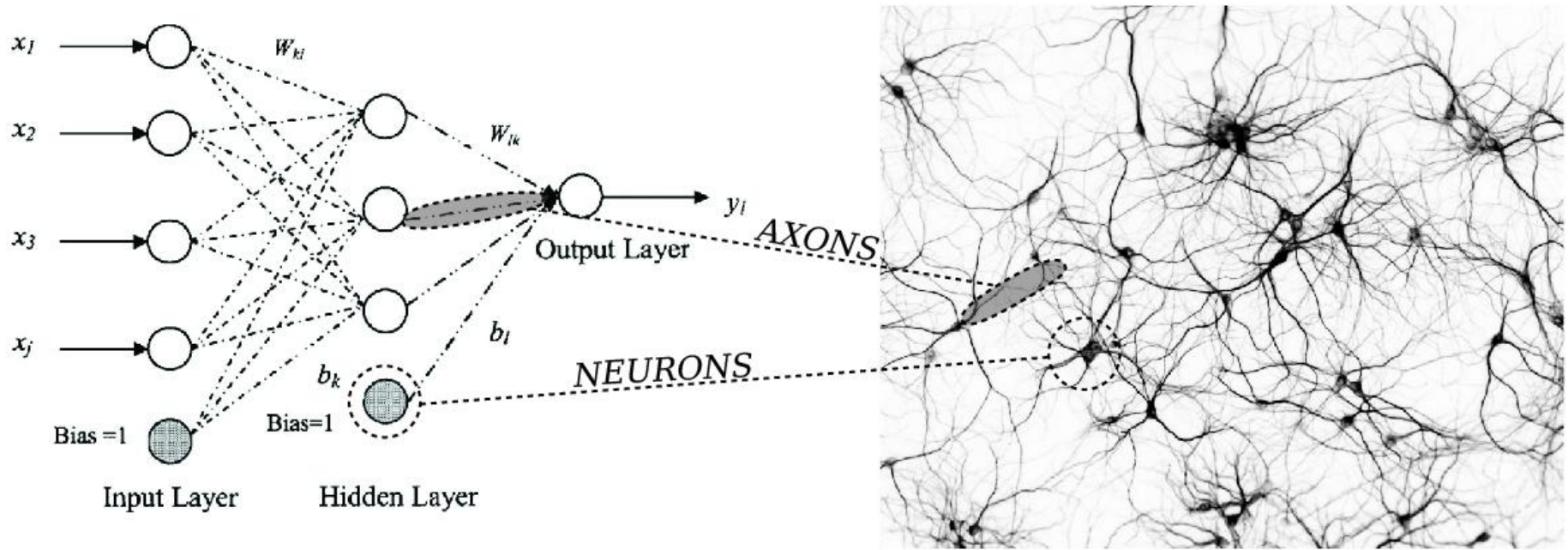
Applicazioni Tipiche



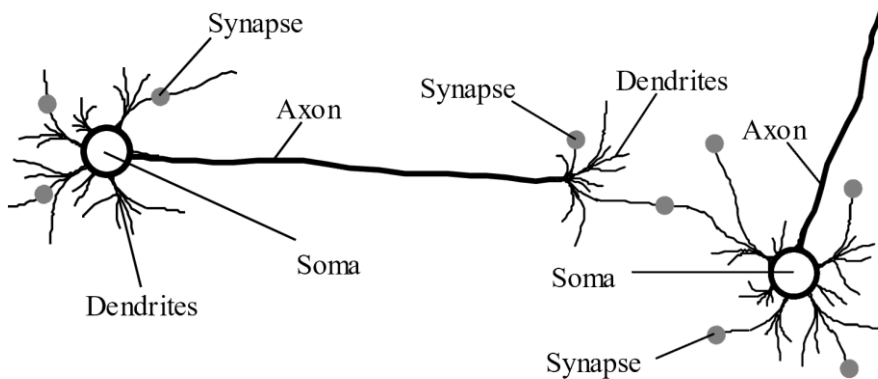
Neural Networks

- L'ispirazione originale era il cervello umano; l'enfasi ora è sull'utilità come strumento computazionale

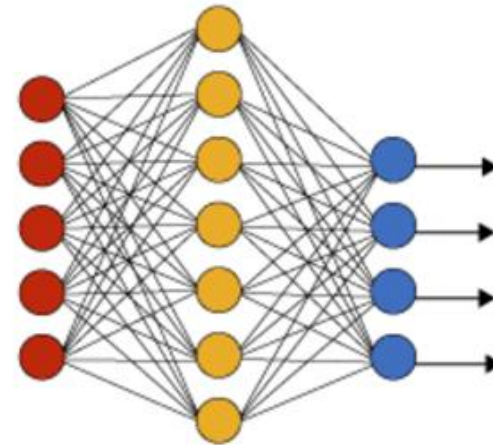
NEURAL NETWORK MAPPING



Parallelismo (II)



Simple Neural Network



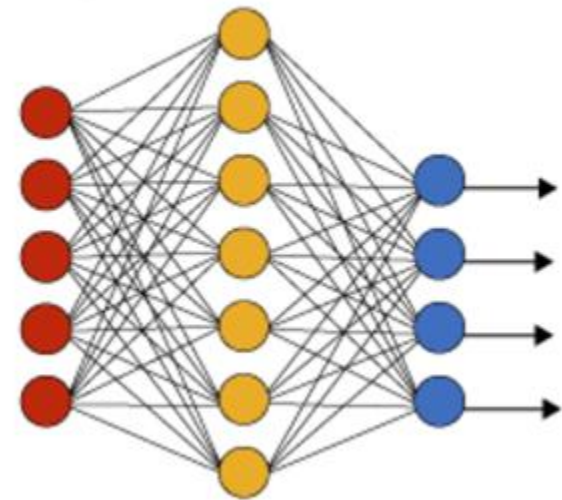
Biological Neural Network	Artificial Neural Network
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

Neural Networks

Feed-forward Network:

- Una struttura a strati composta da un numero di elementi di elaborazione omogenei e semplici (di solito non lineari)
- Tutta l'elaborazione è locale a un elemento di elaborazione ed è asincrona

Simple Neural Network



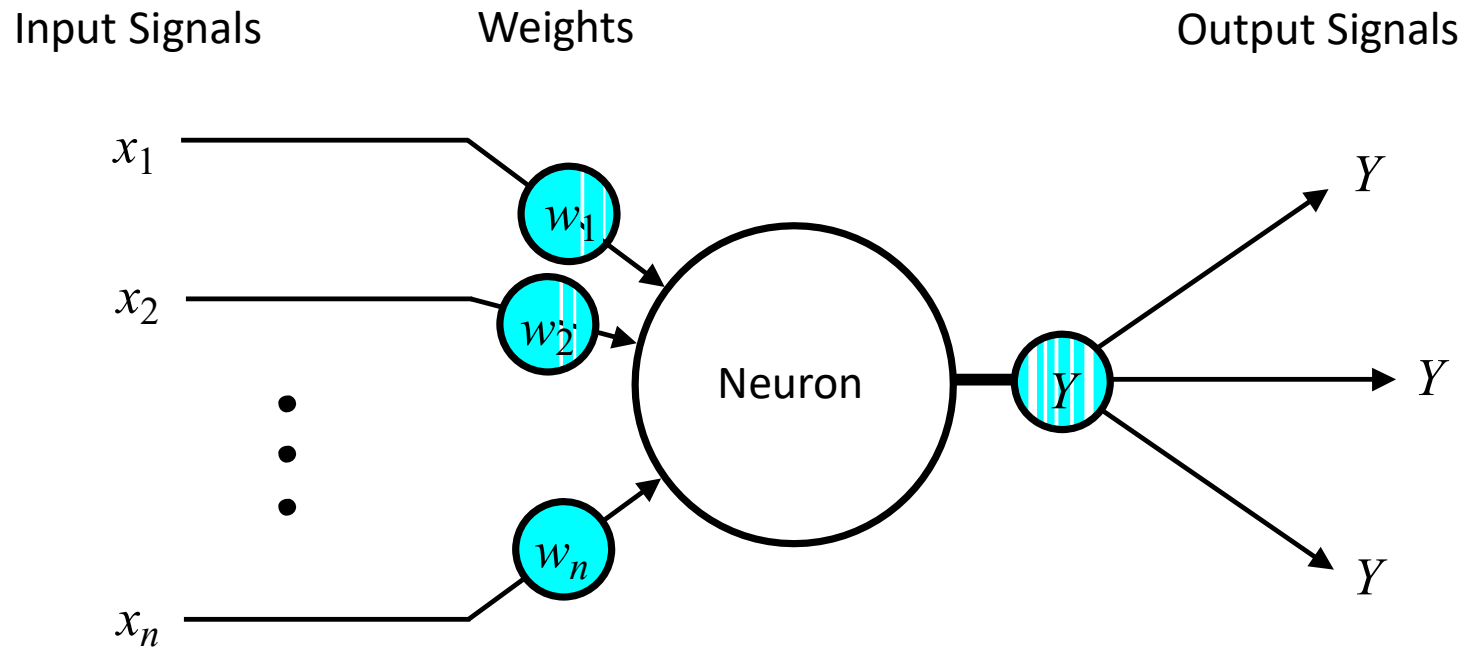
● Input Layer

● Hidden Layer

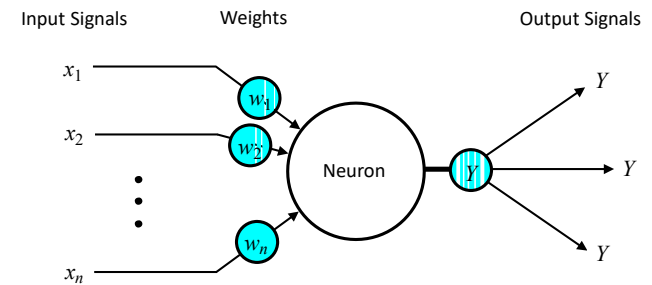
● Output Layer

Il neurone come semplice unità di computazione

Diagramma di un neurone



Modello del neurone



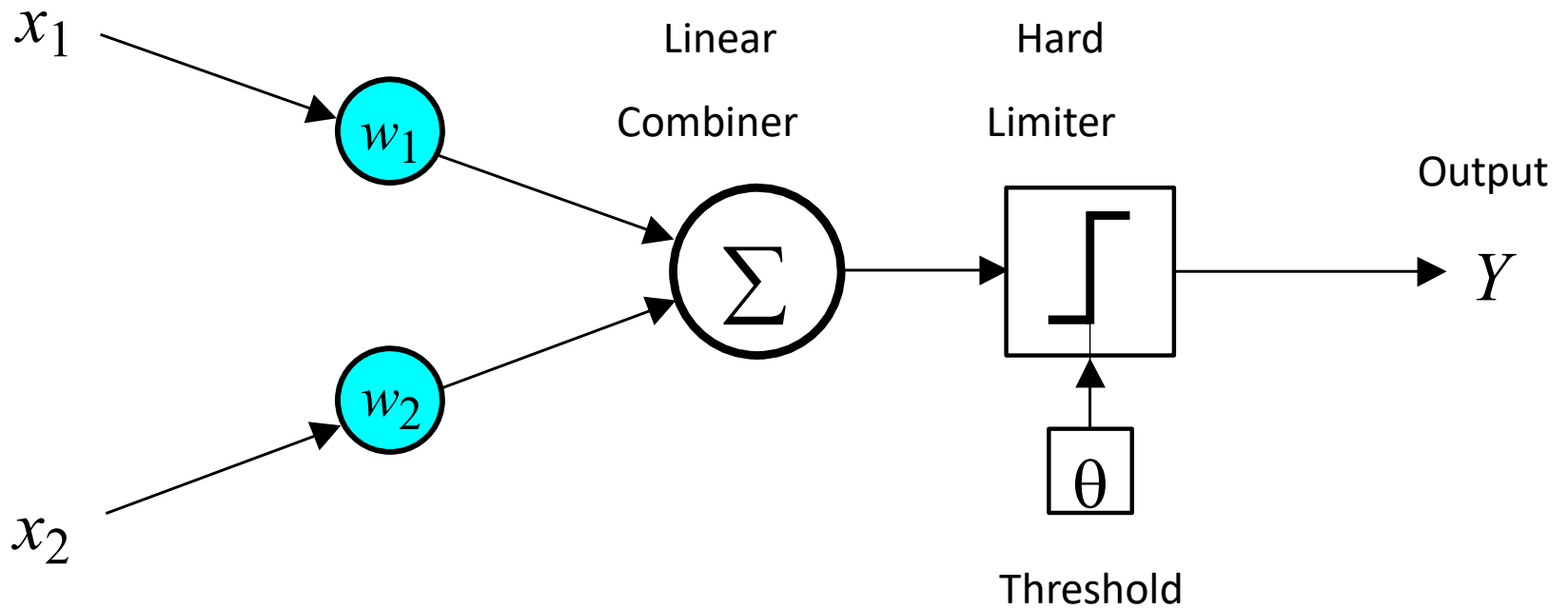
- Il neurone calcola la somma ponderata dei segnali in ingresso e confronta il risultato con un valore di soglia, ϑ .
- Se l'input netto è inferiore alla soglia, l'output del neurone è -1. Ma se l'input netto è maggiore o uguale alla soglia, il neurone diventa attivato e il suo output raggiunge un valore di +1.

$$X = \sum_{i=1}^n x_i w_i \quad Y = \begin{cases} +1, & \text{if } X \geq \theta \\ -1, & \text{if } X < \theta \end{cases}$$

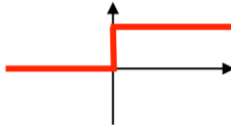
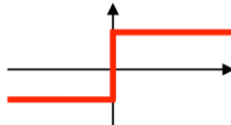
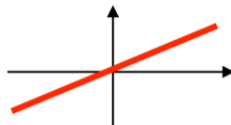
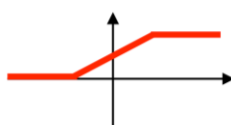
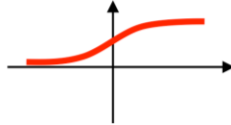
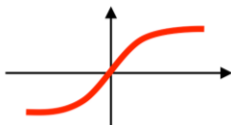
- Questo tipo di funzione di attivazione $Y(X)$ è chiamato **sign function**.

Single-layer two-input perceptron

Inputs

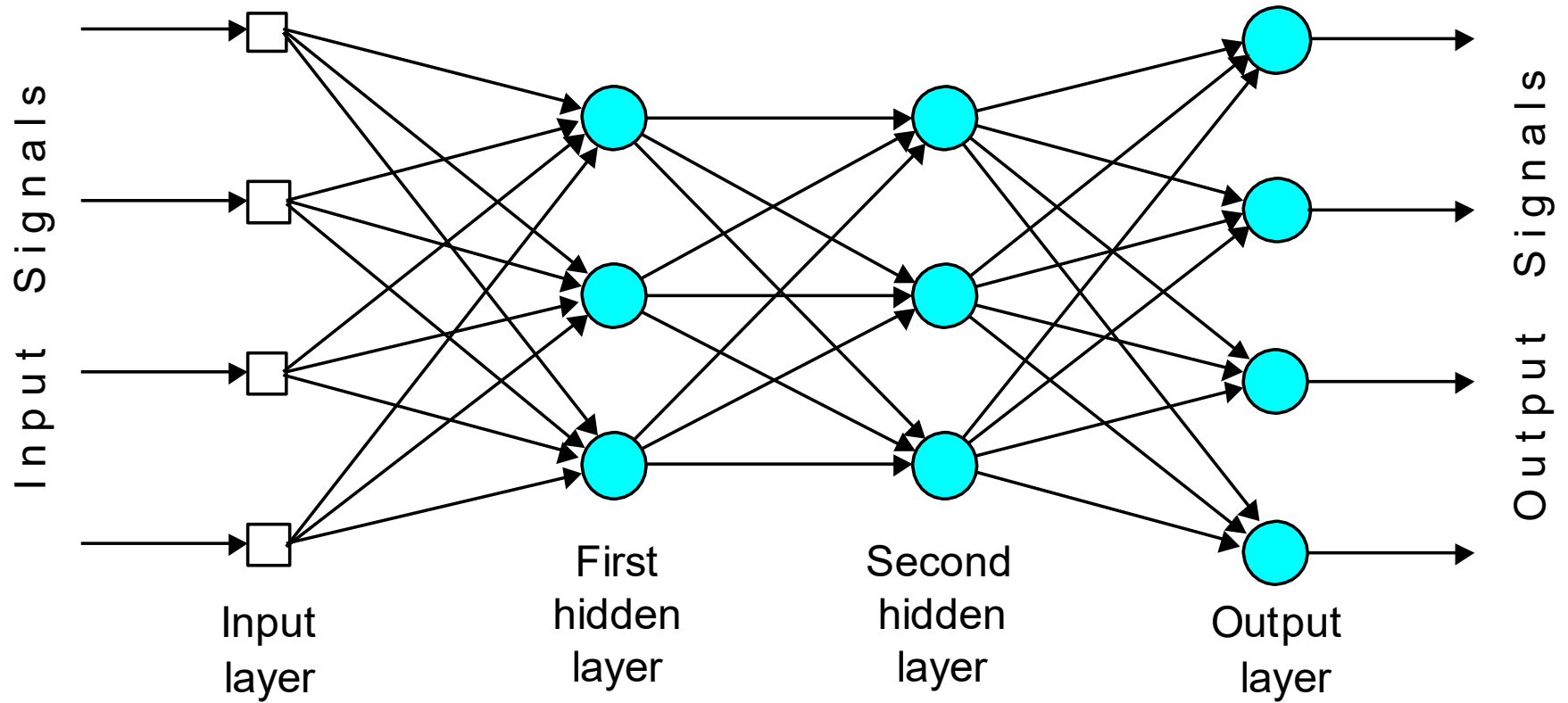


Activation functions

Activation function	Equation	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	
Linear	$\phi(z) = z$	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	

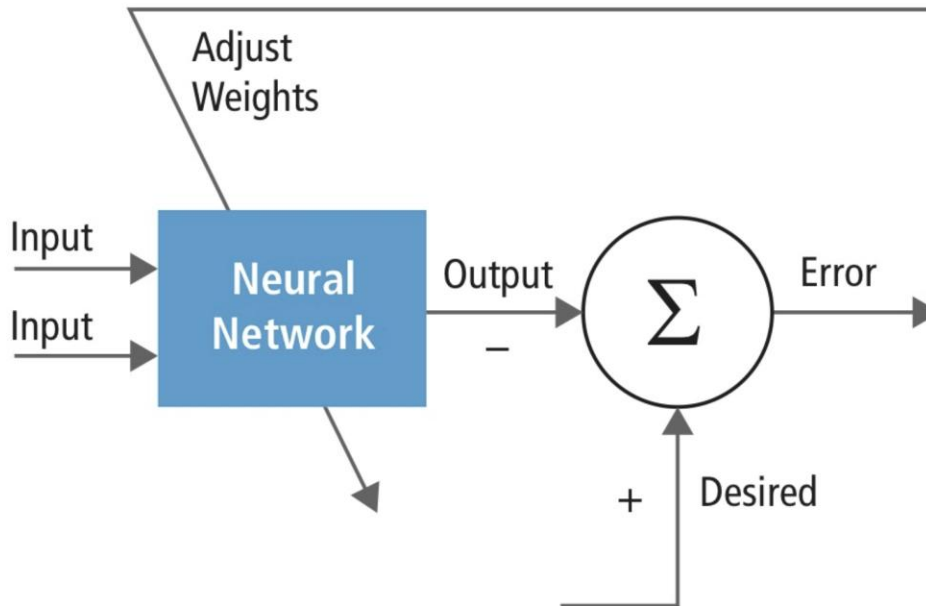
Ogni tipo di funzione di attivazione può modificare il comportamento della propagazione della informazione e del training complessivo

Multilayer perceptron with two hidden layers



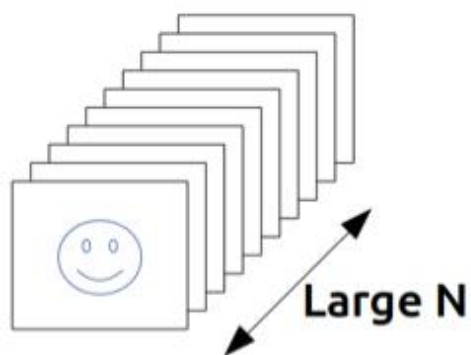
Neural Networks training

- Durante l'addestramento, la Rete Neurale è costretta ad aggiustare i suoi parametri in modo che la sua risposta ai dati in ingresso diventi più vicina alla risposta desiderata.

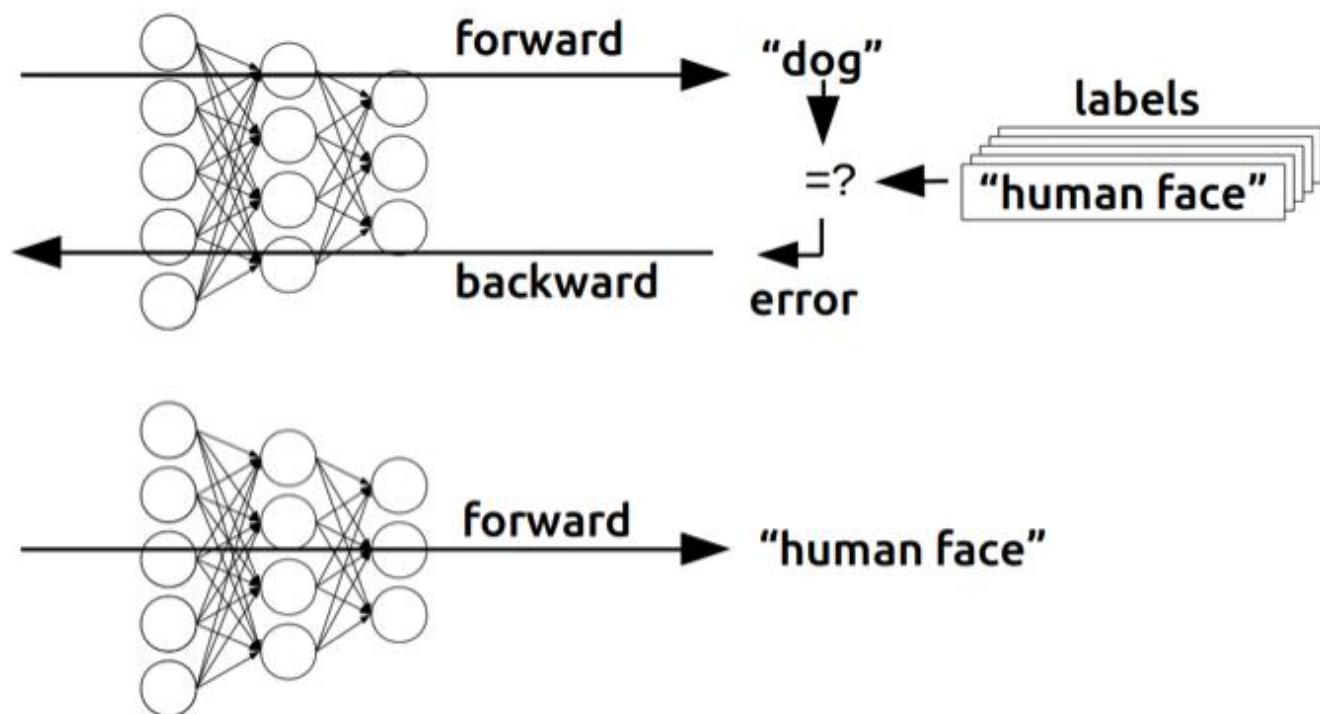
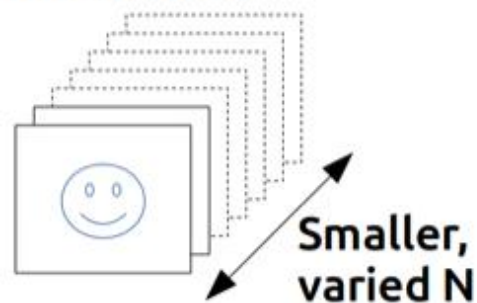


Neural Networks training (2)

Training



Inference



Mai aggiungere
feature senza
attenta
valutazione!

Curse of Dimensionality

Premessa:

- avere in ingresso immagini richiede reti molto grandi
- Ogni parametro della rete deve essere fissato usando i dati
- Servono molti dati per fissare i parametri, anche milioni!

La maledizione è che aumentando gli ingressi aumenta la dimensionalità dello spazio delle feature e i pochi dati disponibili sono molto lontani fra loro

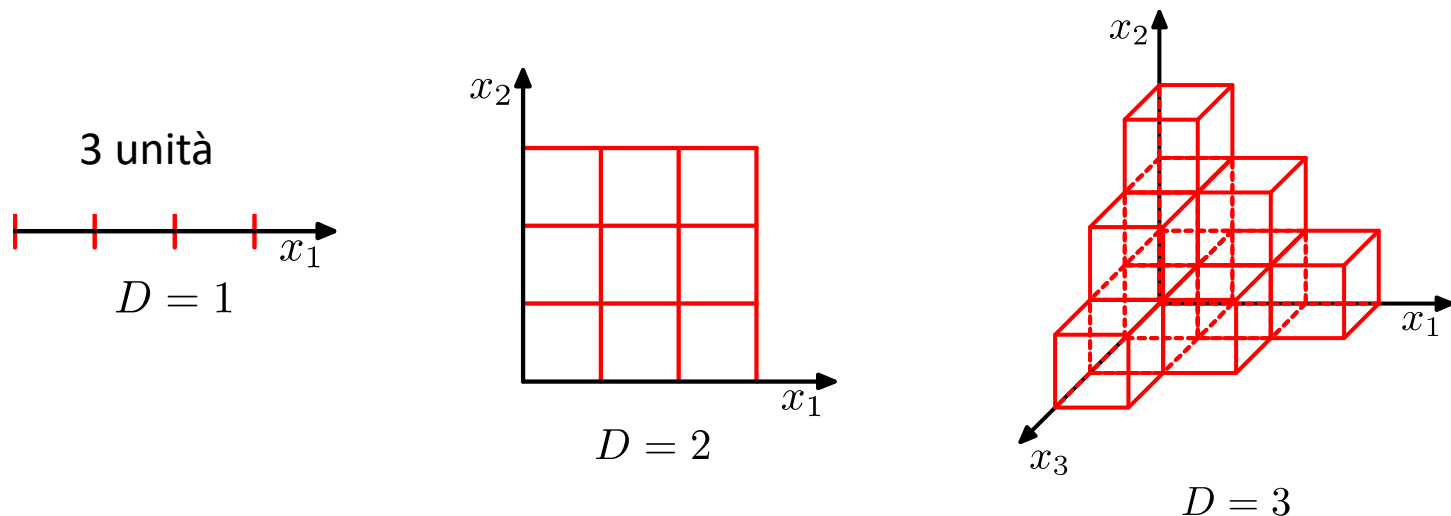
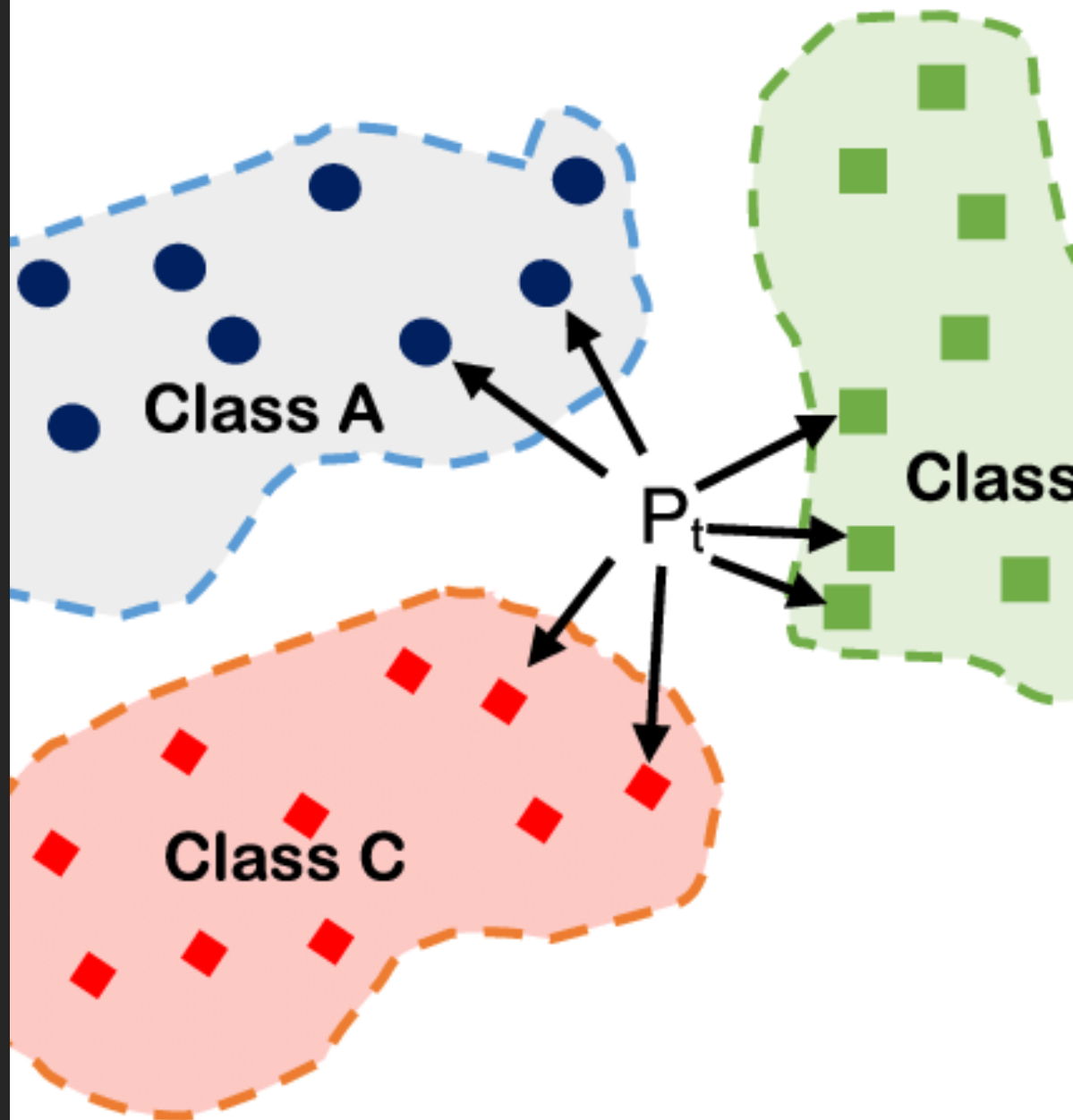


Figure Bishop 1.21

Metodi
tradizionali:
K-nearest
neighbors

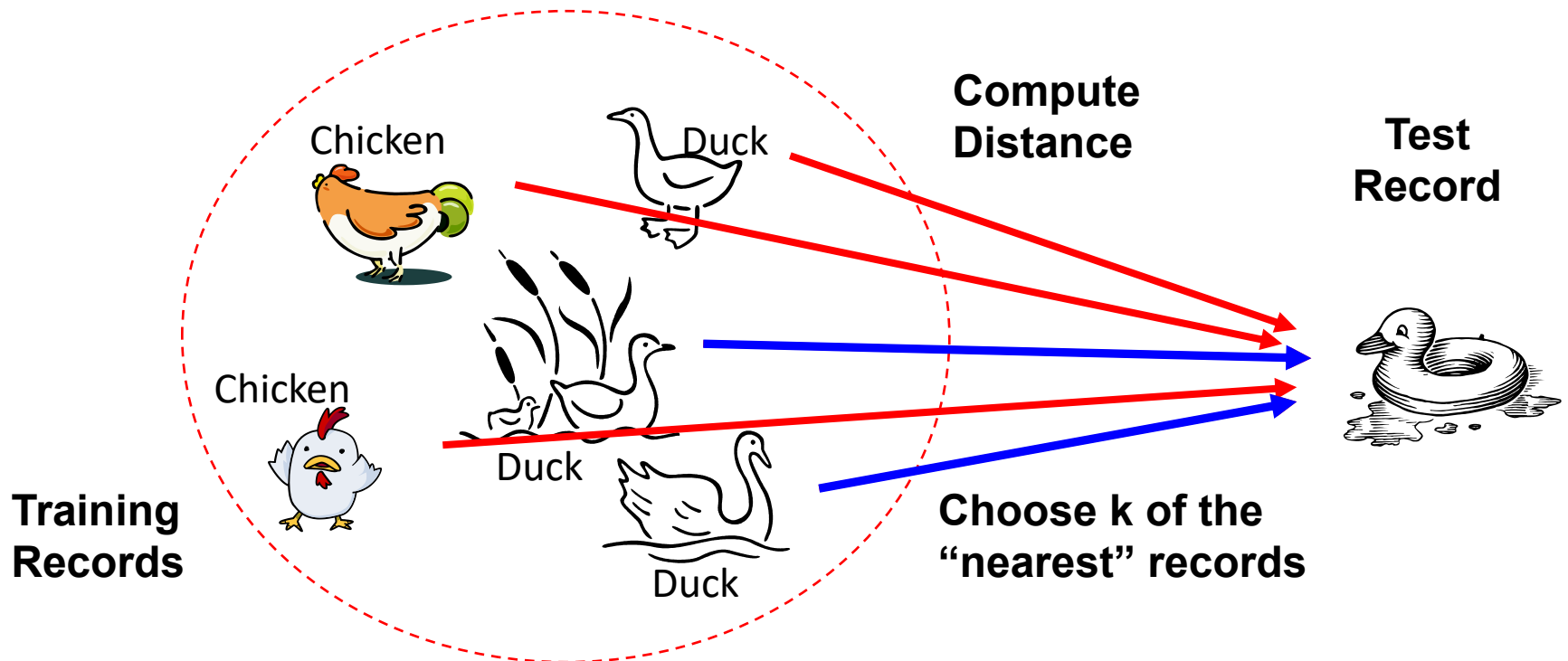
Induttivi ma non CI



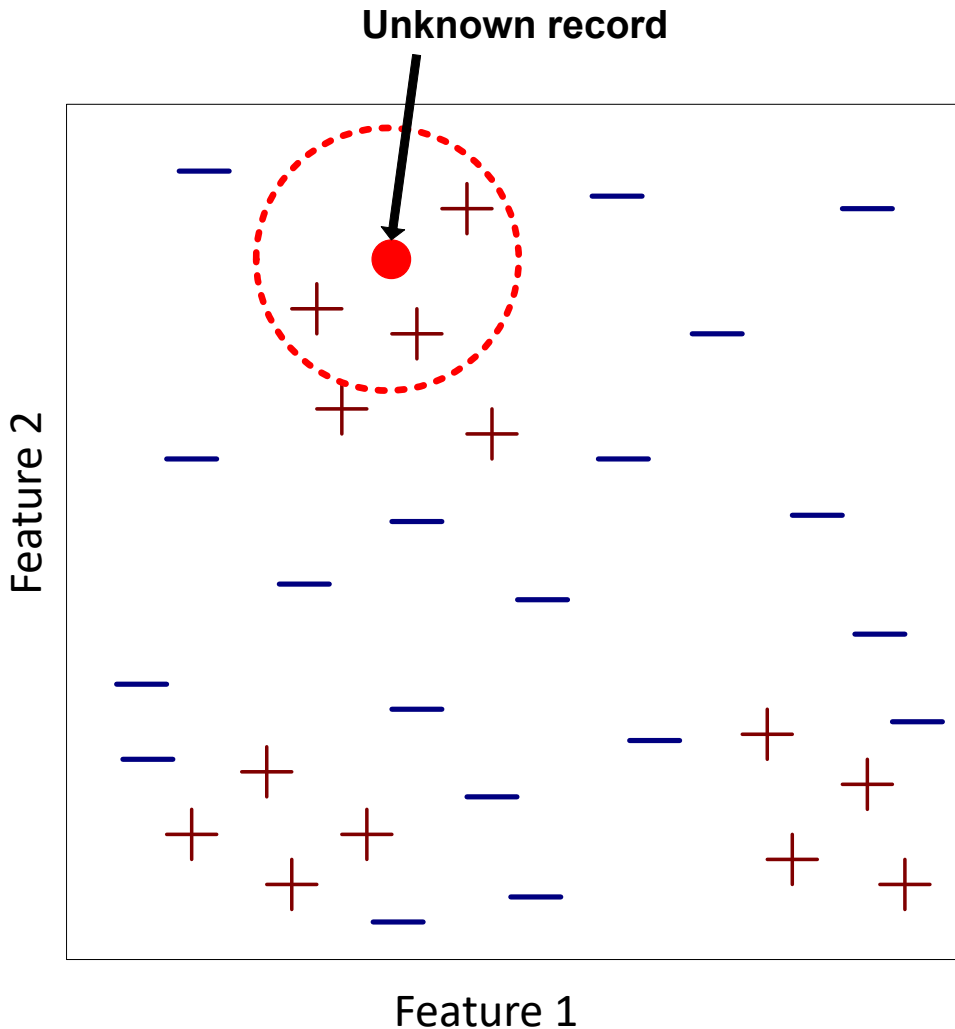
Nearest Neighbor Classifier

Basic idea:

If it walks like a duck, quacks like a duck,
→ then it's probably a duck



Nearest-Neighbor Classifiers



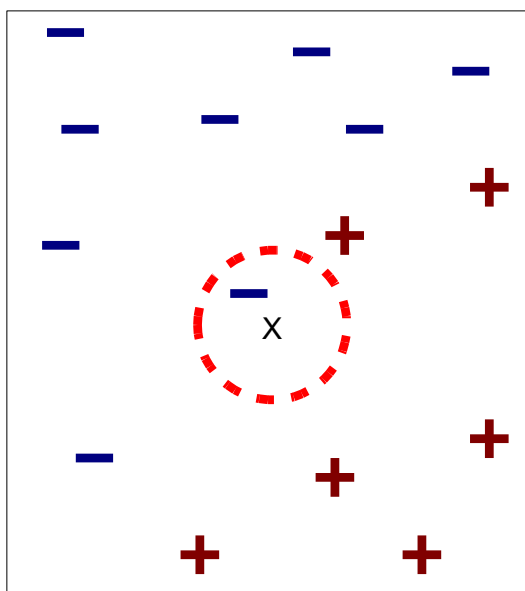
Richiede 3 cose:

- L'insieme dei record memorizzati
- Metrica di distanza per calcolare la distanza tra i record
- Il valore di k , il numero di vicini più prossimi da recuperare

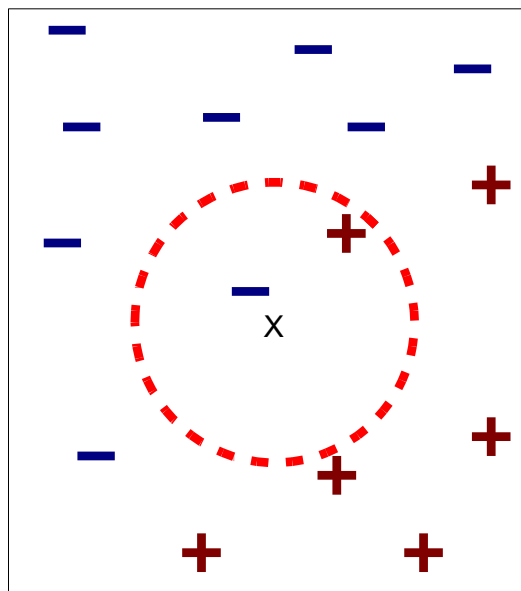
Per classificare un record sconosciuto:

- Calcola la distanza rispetto agli altri record di addestramento
- Identifica i k vicini più prossimi
- Utilizza le etichette di classe dei vicini più prossimi per determinare l'etichetta di classe del record sconosciuto (ad esempio, prendendo il voto di maggioranza)

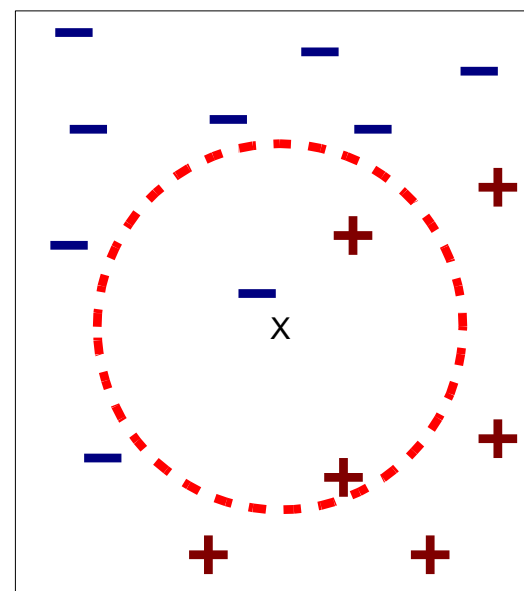
Definizione di primo vicino



(a) 1-nearest neighbor



(b) 2-nearest neighbor



(c) 3-nearest neighbor

I k vicini più prossimi di un record 'x' sono i punti dati che hanno la distanza più piccola rispetto a x.

Nearest Neighbor Classification

1) Calcola la distanza tra due punti: Esempio: Distanza euclidea

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$$

2) Determina la classe dalla lista dei vicini più prossimi

A) Prendi il voto di maggioranza delle etichette di classe tra i k vicini più prossimi

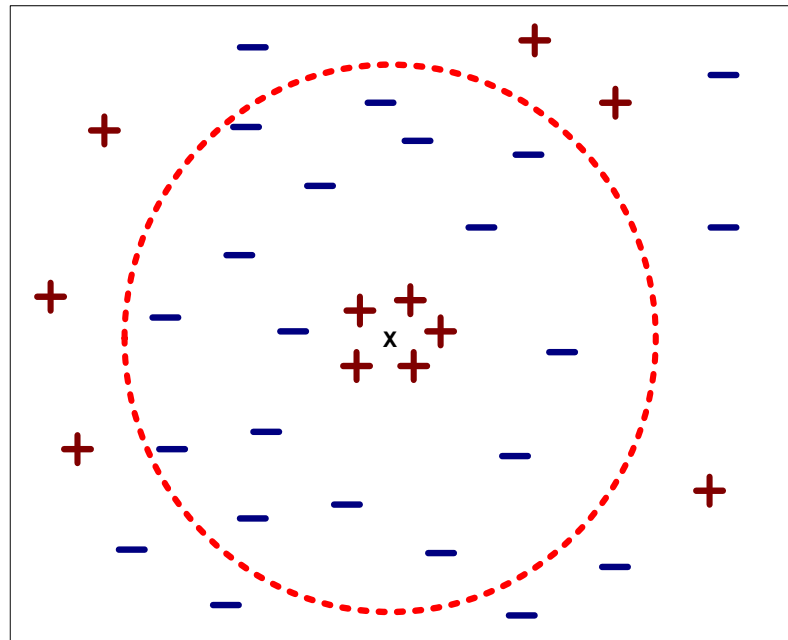
B) OPZIONALE: Pesa il voto in base alla distanza fattore di peso, $w=1/d^2$

Nearest Neighbor Classification...

Scegliere il valore di k :

- Se k è troppo piccolo, sensibile ai punti di rumore
- Se k è troppo grande, il vicinato potrebbe includere punti di altre classi

Il campione incognito 'x'
per k piccoli viene classificato come '+'
per k medi viene classificato come '-'
per k grandi può cambiare ancora...



kNN: problema con le scale degli ingressi

Problemi di scalabilità:

- Gli attributi potrebbero dover essere scalati/normalizzati per evitare che le misure di distanza siano dominate da uno degli attributi Esempio con 3 input:
 - l'altezza di una persona può variare da 1,5 m a 1,8 m
 - il peso di una persona può variare da 90 lb a 300 lb
 - il reddito di una persona può variare da \$10.000 a \$1.000.000

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$$

- Le reti neurali tendono ad auto-scalare gli ingressi.

kNN: sono lazy learners (pigri!)

- **k-NN**

- Non creano un modello esplicito
- Classificare un ingresso è costoso come
 - Memoria
 - Computational complexity

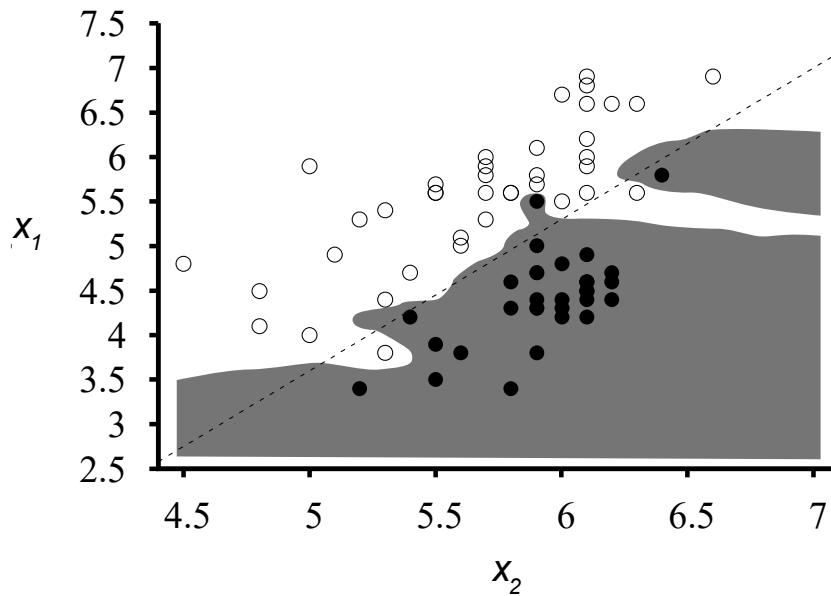
- Eager learners

- Creano un modello compatto e più sintetico dei dati (learning più complesso)
- Esempi
 - Decision tree
 - Rule-based systems
 - Reti Neurali

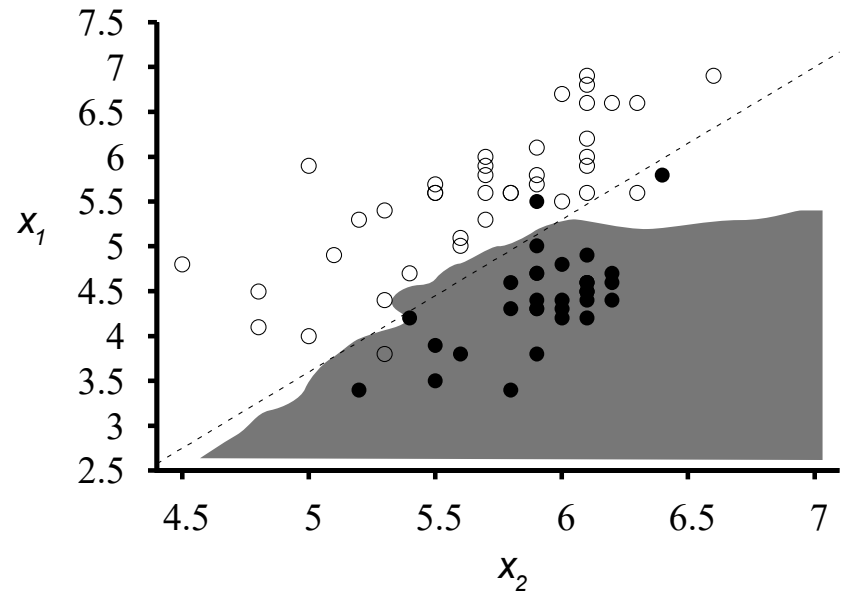


Overfitting e Underfitting

- *k troppo piccolo* → overfitting
- *k troppo grande* → underfitting



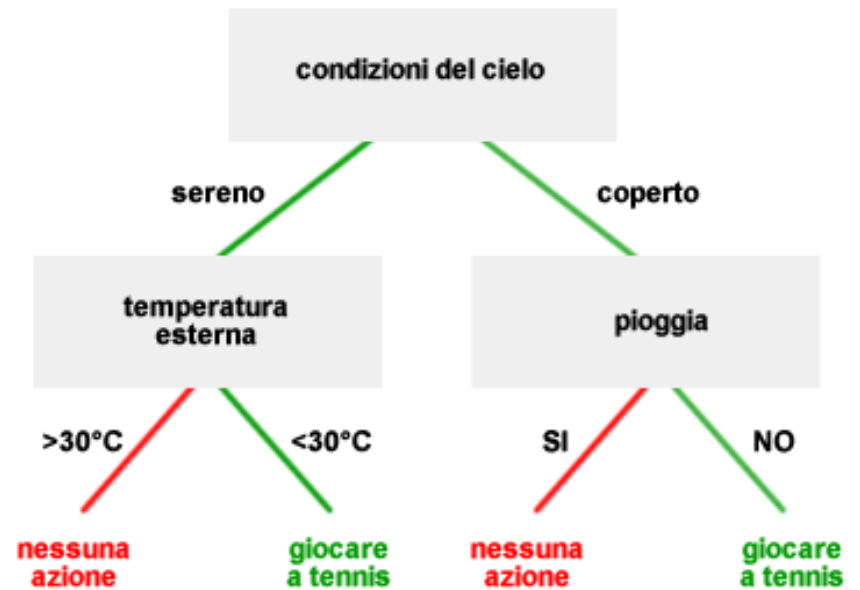
$k = 1$



$k = 5$

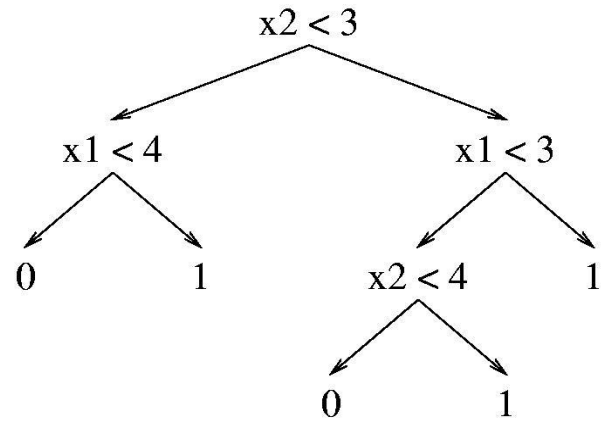
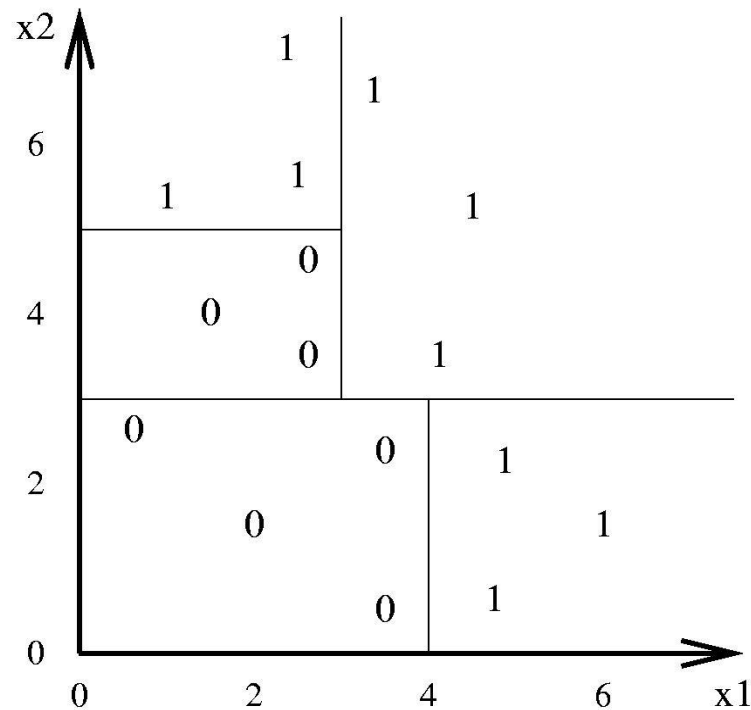
Metodi classici: Decision tree

Induttivi ma non CI



Decision Tree Decision Boundaries

Decision trees divide the feature space into axis-parallel rectangles, and label each rectangle with one of the K classes.

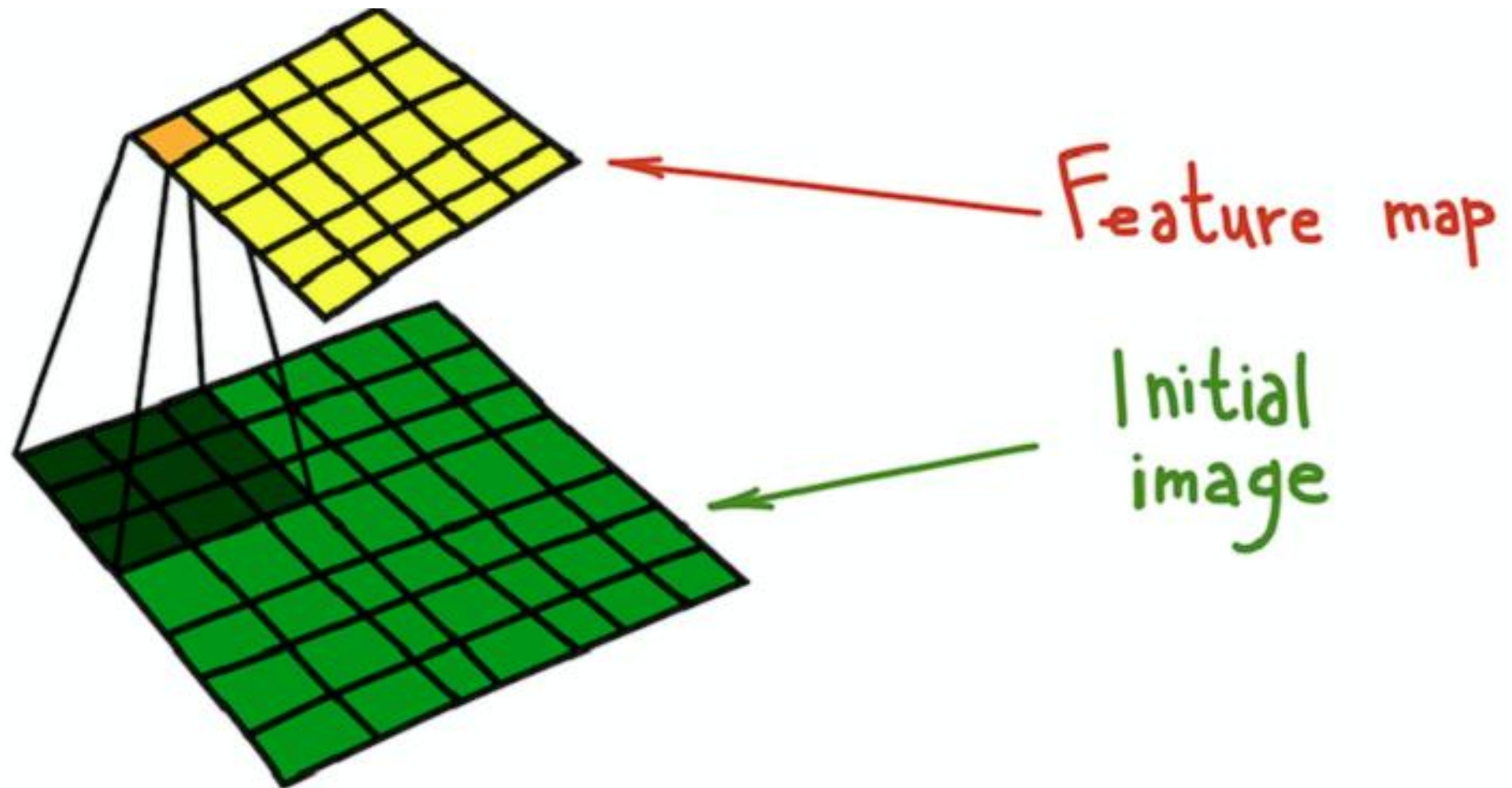


ALBERI DECISIONALI
Sono strutture di IF
con soglie

Al termine dell'apprendimento

- Indipendentemente dal modello di ML scelto, al termine dell'apprendimento avremo solo un blocco di codice completo ed usabile.
- Avendo un nuovo dato/segnale/immagine X possiamo immaginare di scrivere

```
Classe_stimata1 = ReteFeedForward(X);  
Classe_stimata2 = 5_NN (X);  
Classe_stimata3 = albero (X);
```

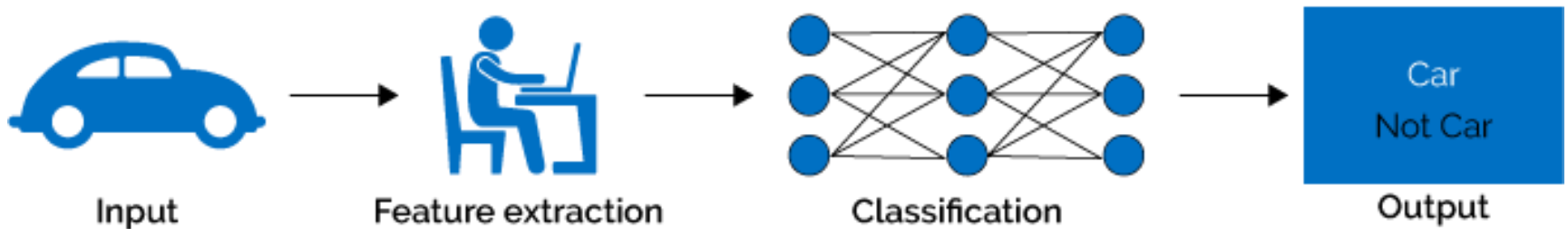



Feature di diverso livello

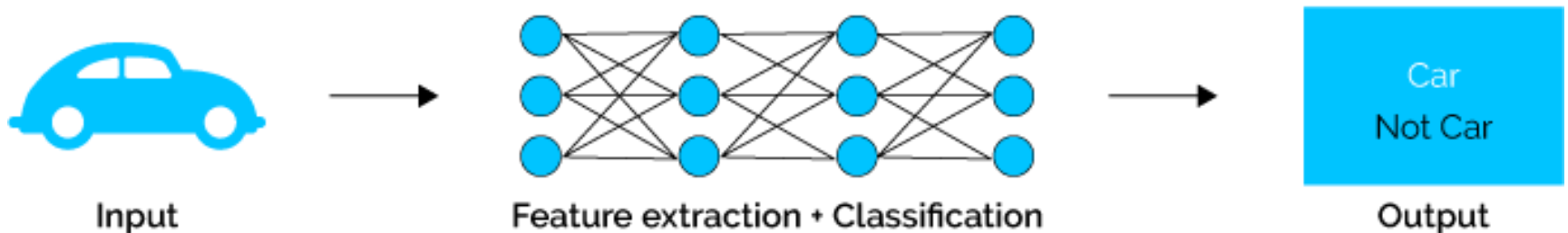
Cosa serve per la parte iniziale di un processo di machine learning

ML vs Deep Learning

Machine Learning



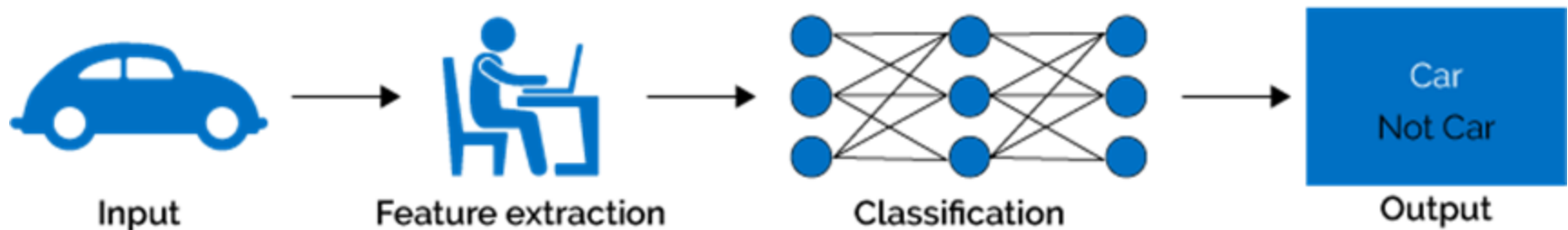
Deep Learning



Focalizziamoci su ML Trad.

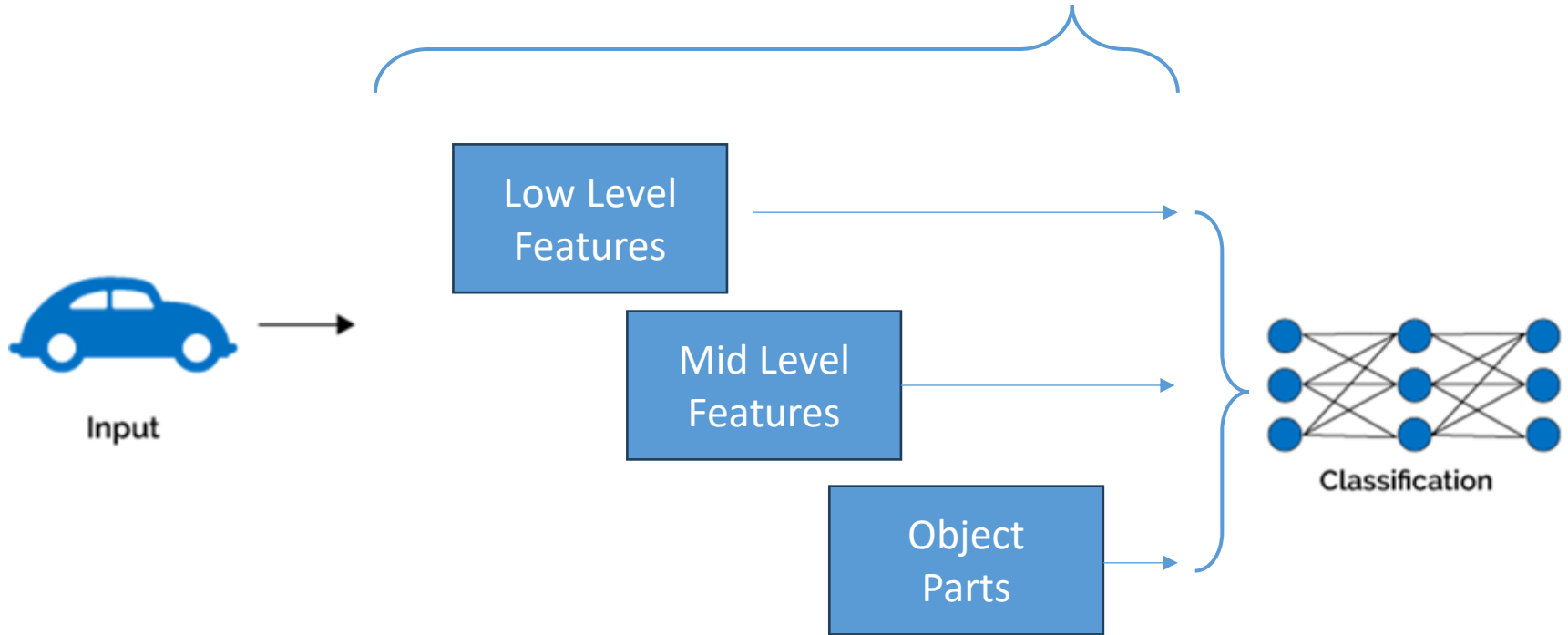
- Vediamo come lavora il designer della applicazione in modo tradizionale
- Questo ci permetterà di capire come funzionano le reti Deep che vanno appunto ad automatizzare alcuni passaggi del designer

Machine Learning



Human Designer

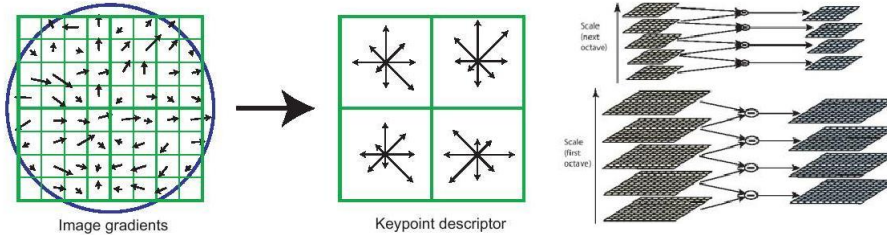
Feature extraction



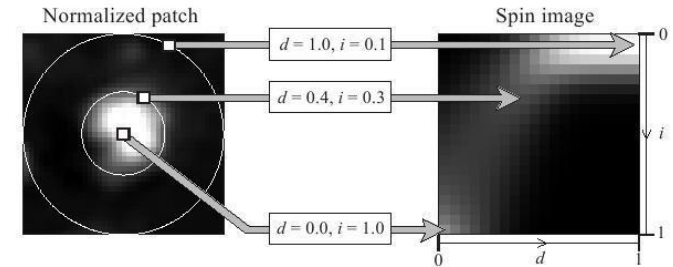
Designer Feature extraction



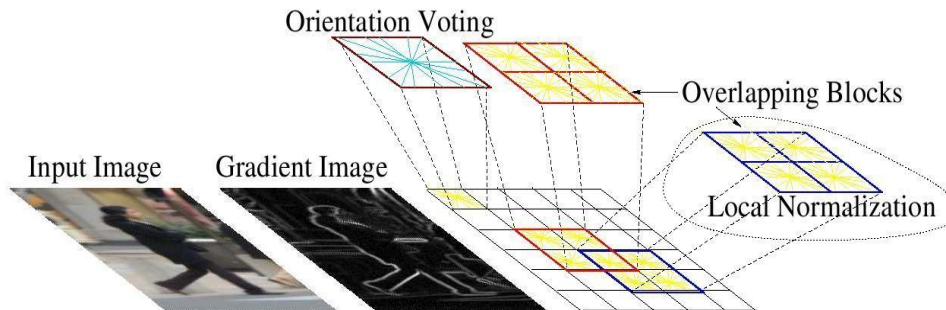
• Low-level features



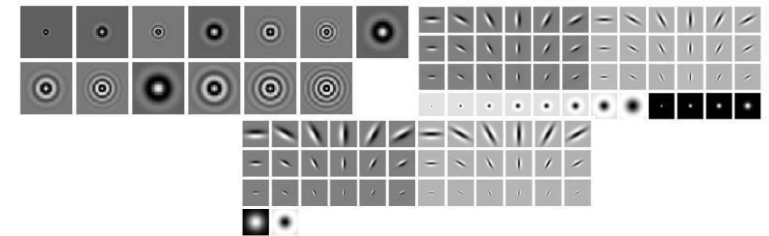
SIFT



Spin image
(Kernels)



HoG



Textons
(Kernels)

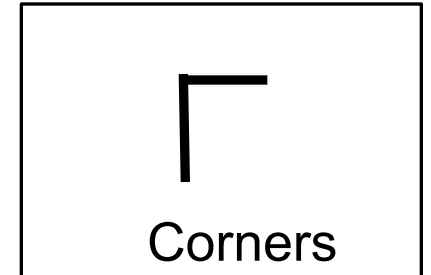
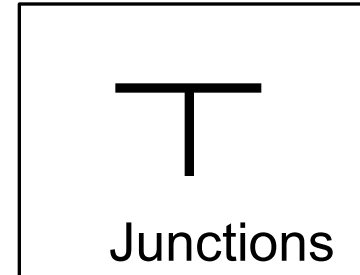
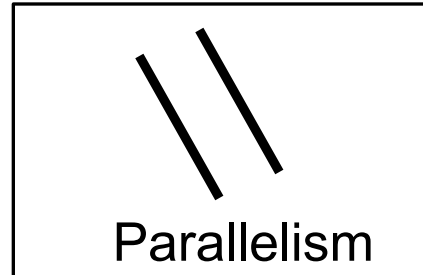
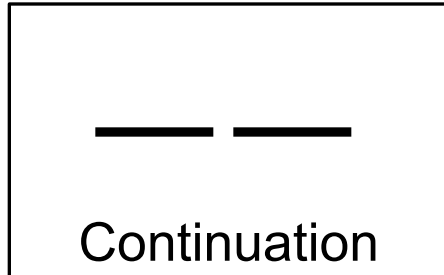
and many others:

SURF, MSER, LBP, Color-SIFT, Color histogram, GLOH,

Designer Mid level representations



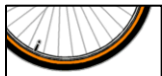
- Mid-level features



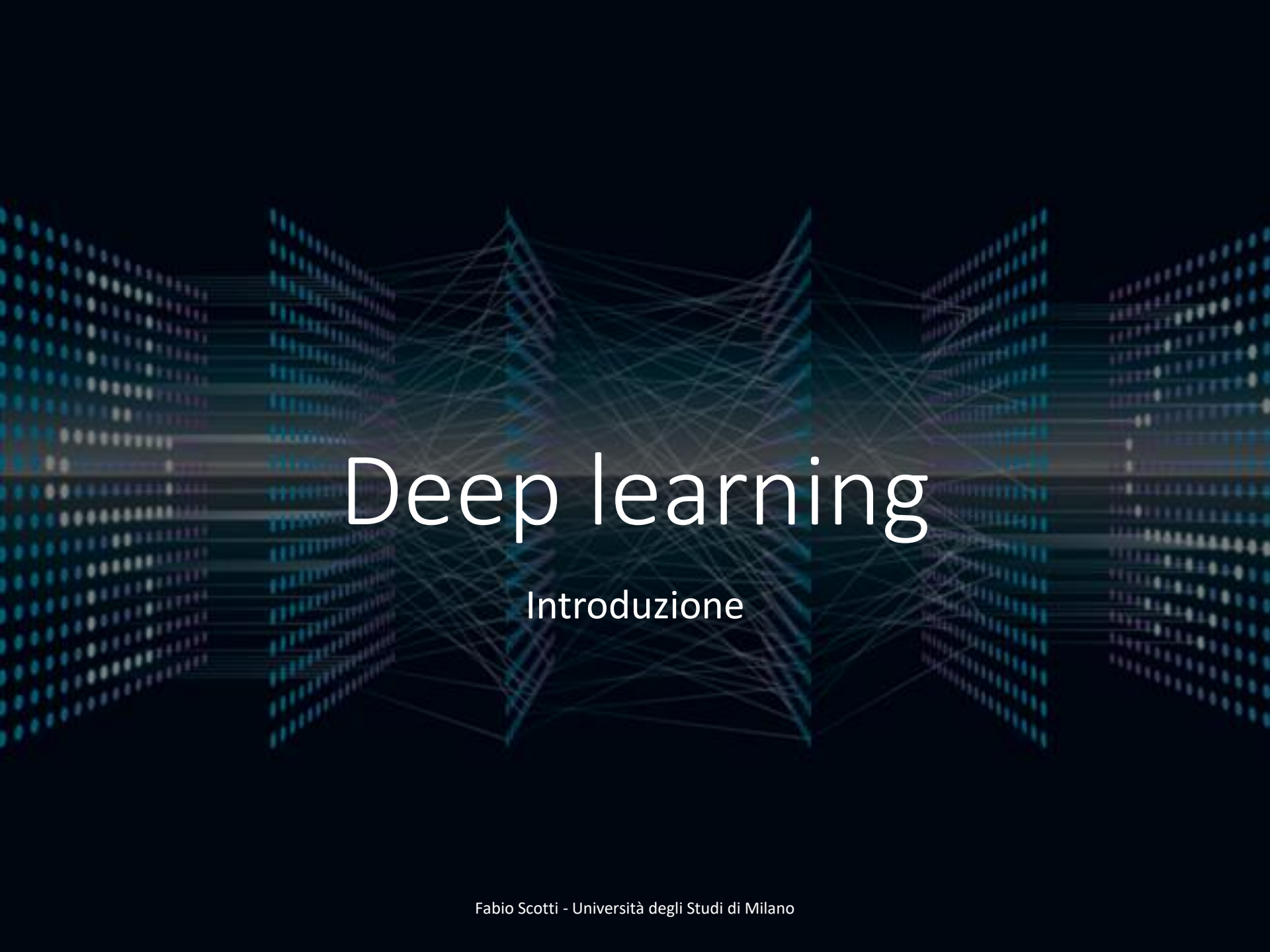
“Tokens” from Vision by D.Marr:



- Object parts:



E' difficile da ingegnerizzare! → I kernel migliori li faremo trovare automaticamente al metodo di apprendimento Deep!

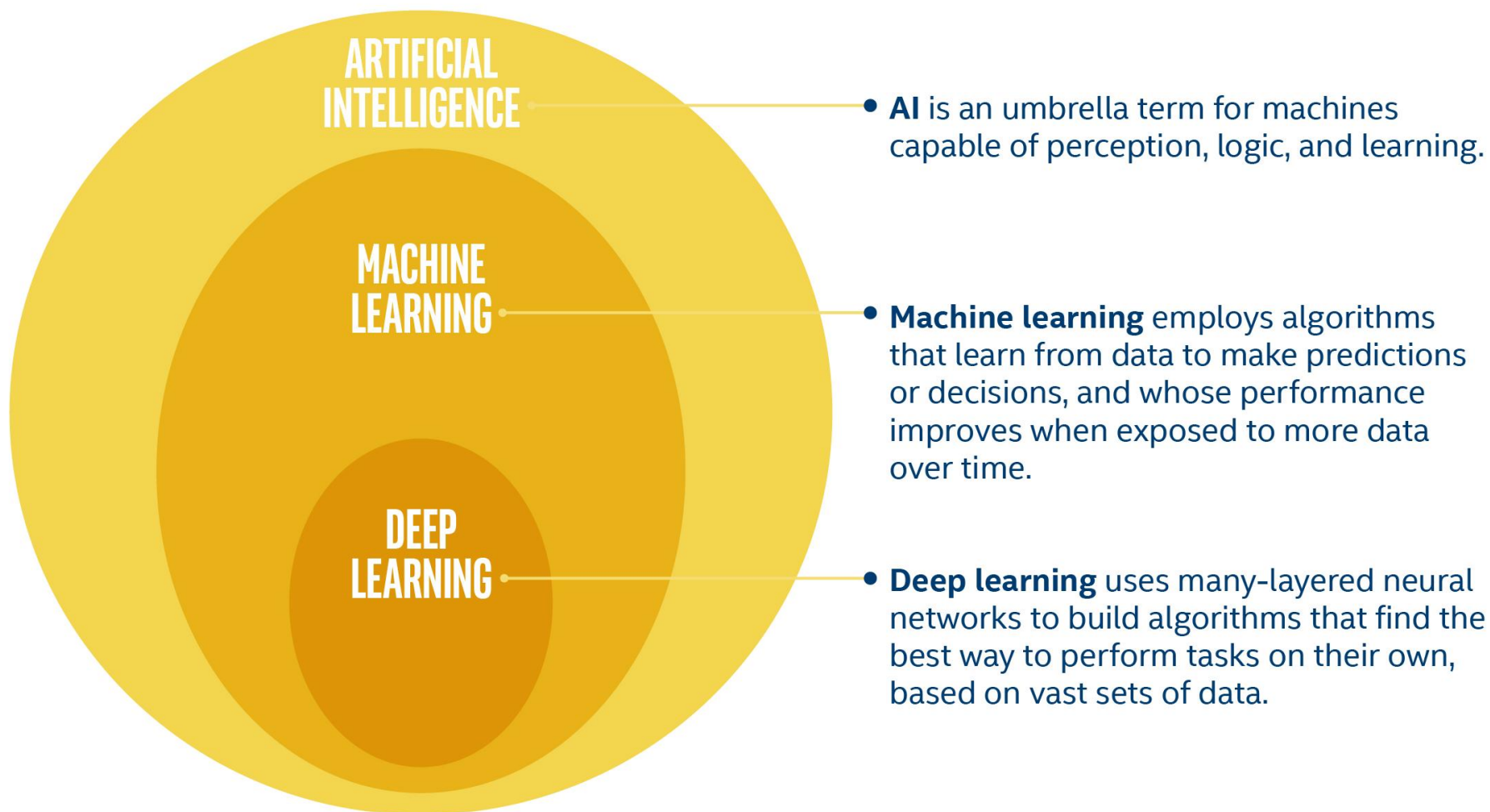
The background is a dark blue gradient with a complex, abstract pattern of glowing blue and white dots and lines, resembling a neural network or data flow. The dots are arranged in a grid-like pattern, and the lines connect them in a web-like structure, creating a sense of depth and connectivity.

Deep learning

Introduzione

DL < ML < AI

(richiamo della lezione precedente)



Il deeplearn non è 1 solo metodo..

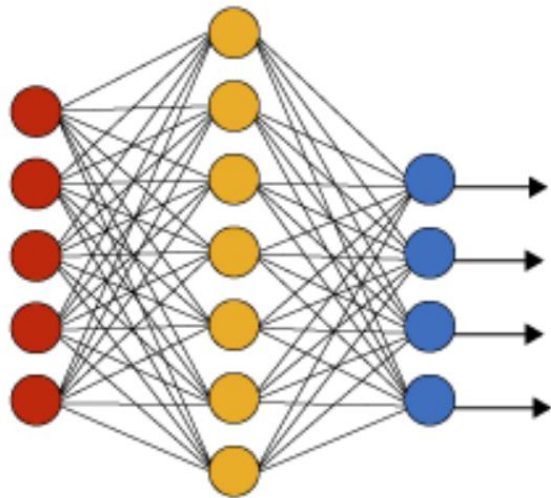
Ma indica una **classe di algoritmi**

1. usano **vari livelli di unità non lineari a cascata** dove ciascun livello successivo utilizza l'uscita del livello precedente come input per svolgere compiti di
 - *estrazione di caratteristiche*
 - trasformazione.
2. Gli algoritmi possono essere sia di tipo
 - **supervisionato** (classificazione, regressione)
 - sia **non supervisionato** (l'analisi di pattern)

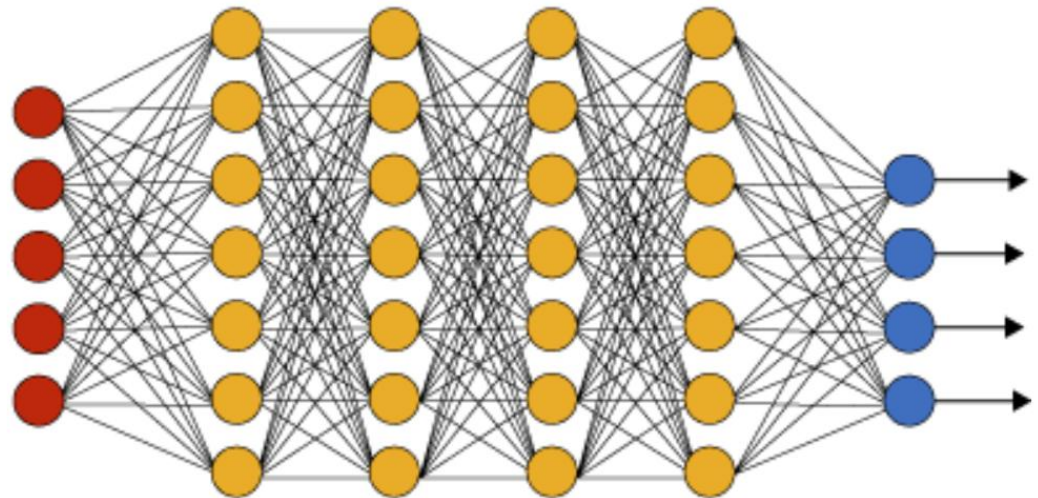
3. apprendimento di livelli gerarchici multipli di caratteristiche (e di rappresentazioni) dei dati
 1. **I livelli non sono espressi dal supervisore ma creati automaticamente dal sistema.**
 2. **Le caratteristiche di più alto livello vengono derivate da quelle di livello più basso** per creare una rappresentazione gerarchica.
4. **livelli multipli** corrispondono a **differenti livelli di astrazione** → formano una gerarchia di concetti.

Shallow and Deep Learning networks

Simple Neural Network



Deep Learning Neural Network



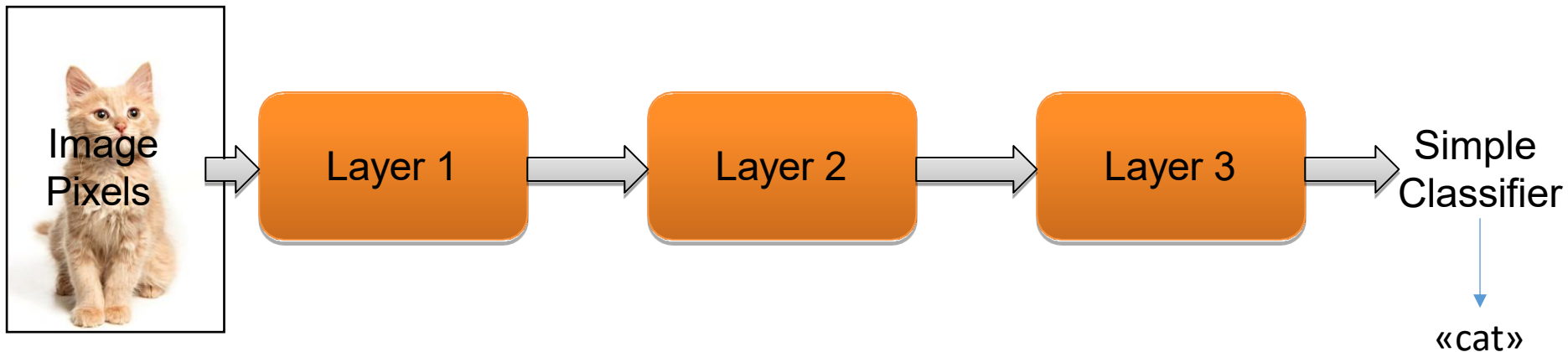
● Input Layer

● Hidden Layer

● Output Layer

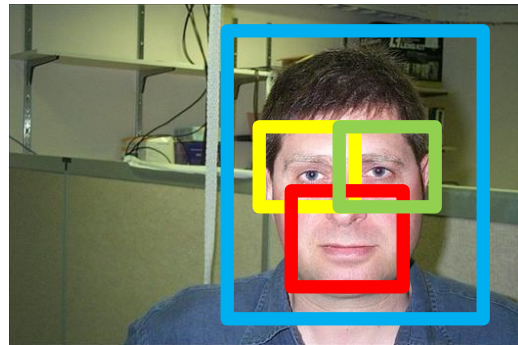
Le reti imparano la gerarchia delle feature

In questo esempio di classificazione di immagini le rete impara ad estrarre le feature necessarie e a far funzionare gli strati successivi dalle uscite degli strati precedenti, di fatto organizzando una gerarchia nelle feature

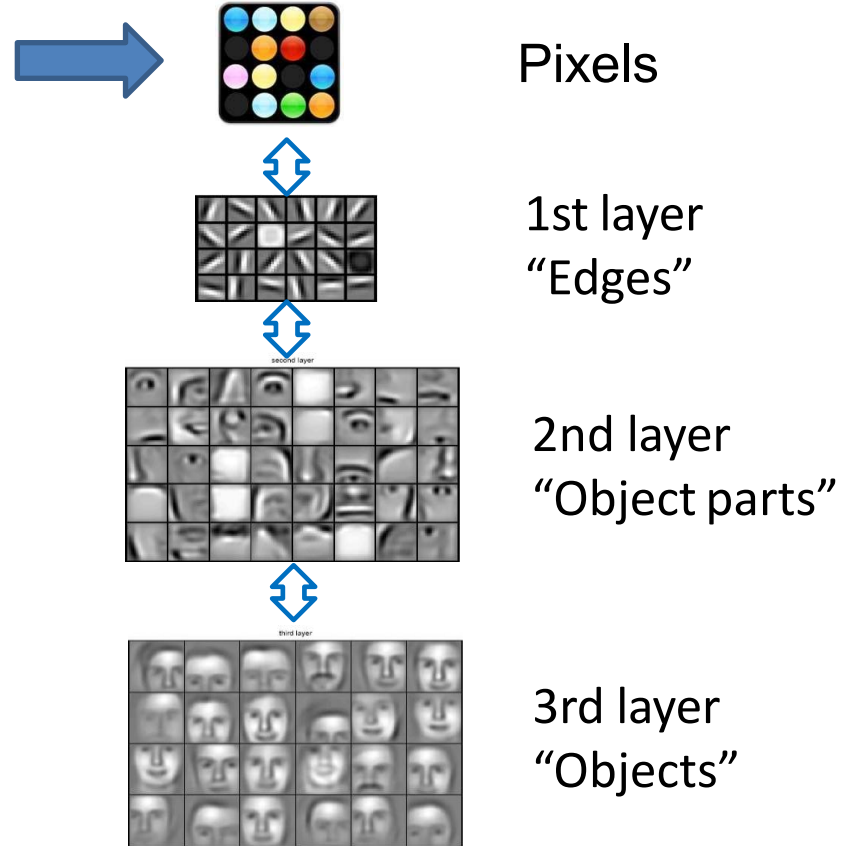


Tutti gli strati vengono allenati insieme

Le reti imparano la gerarchia delle feature: Caso del volto



Input data



Feature representation

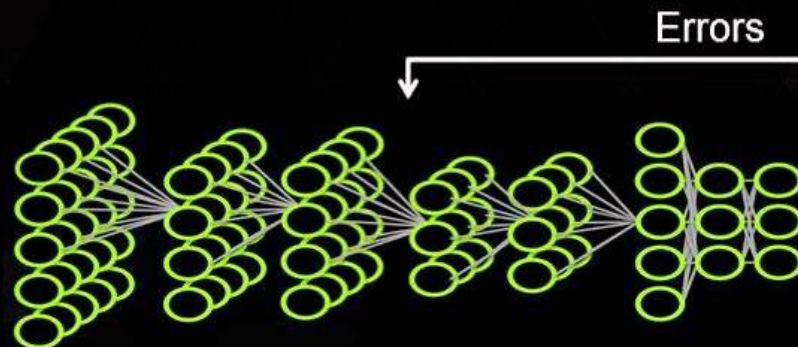


... Resto della rete deep

Inoltre le reti durante
l'apprendimento
fissano i parametri dei filtri usati
per estrarre le feature

Apprendimento nel caso di Deep Learning

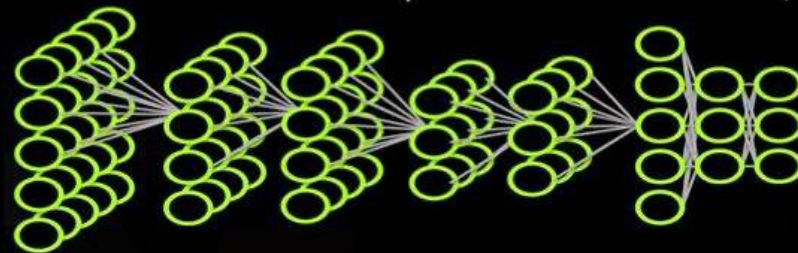
Train:



Dog
Cat
Raccoon



Deploy:



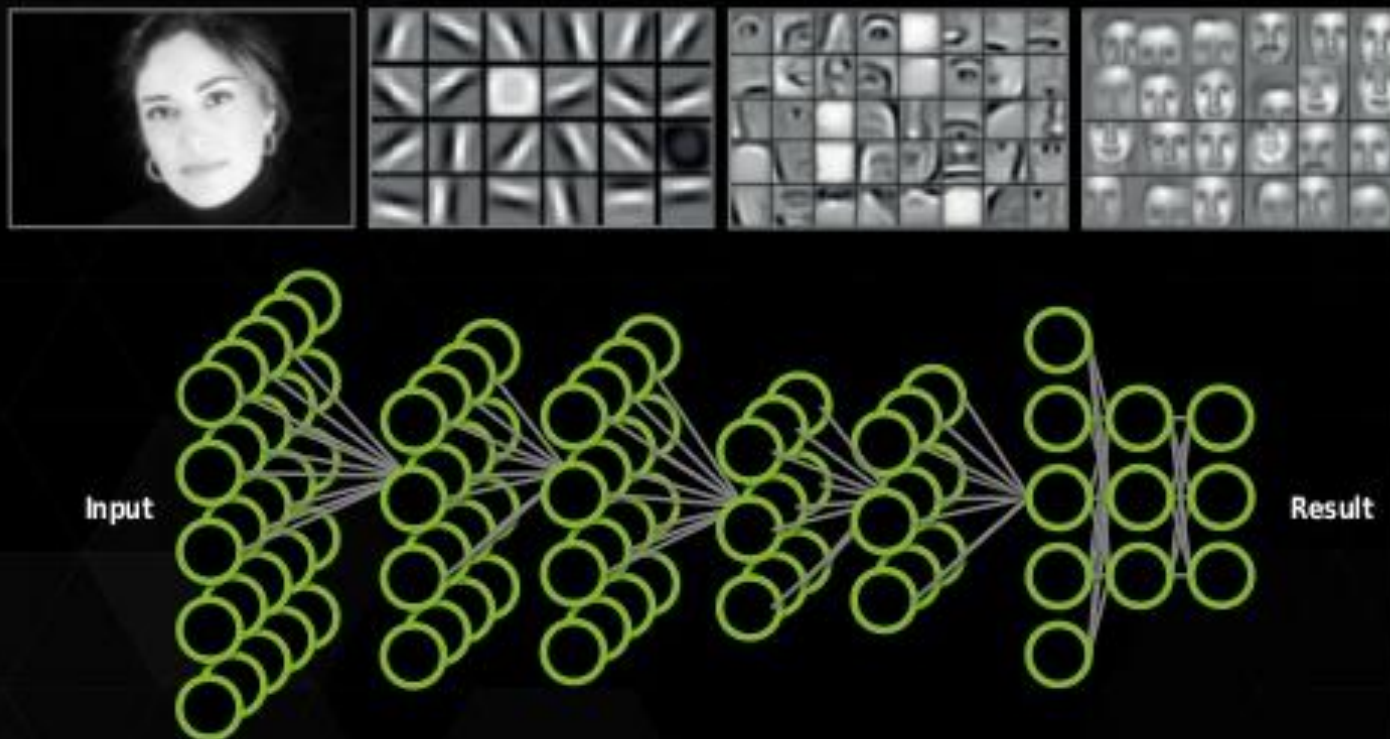
Dog



Il concetto è molto simile a quanto visto per le reti tradizionali

Quando si parla di Deep Learning?

In modo generale si può pensare a questi numeri di riferimento



Reti attuali
considerate deep

~10 layers
1B parameters
10M images
~30 Exaflops
~30 GPU days

Human brain has trillions
of parameters - only
1,000 more.

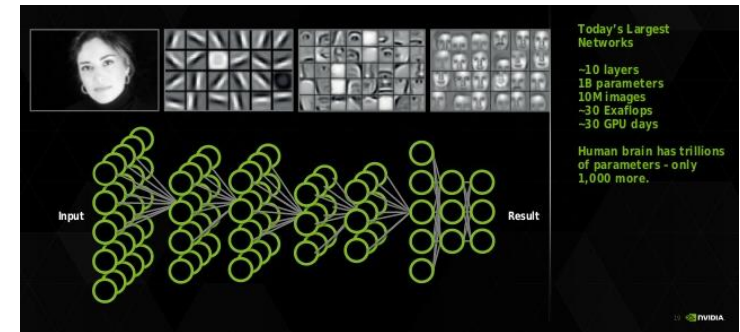
Altri autori parlano di deep quando si hanno 4-5 strati nascosti e centinaia/migliaia di neuroni sugli strati nascosti a seconda delle applicazioni (immagini, traduzione, speech rec.)

Deep Learning tasks

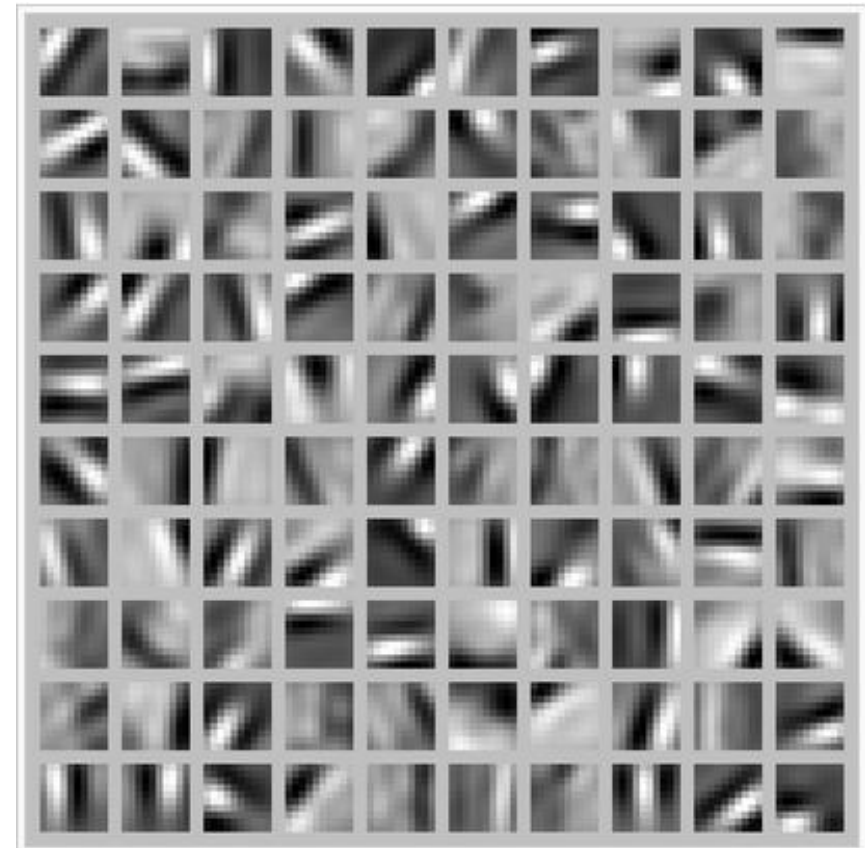
- Di solito è migliore quando lo spazio di input è strutturato localmente, sia spazialmente che temporalmente: immagini, linguaggio, ecc. rispetto a caratteristiche di input arbitrarie.

- Esempio: "vista" di un livello di caratteristiche di visione appreso
 - ogni quadrato nella figura mostra l'immagine di input che attiva al massimo una delle 100 unità

Adapted from © Tony Martinez



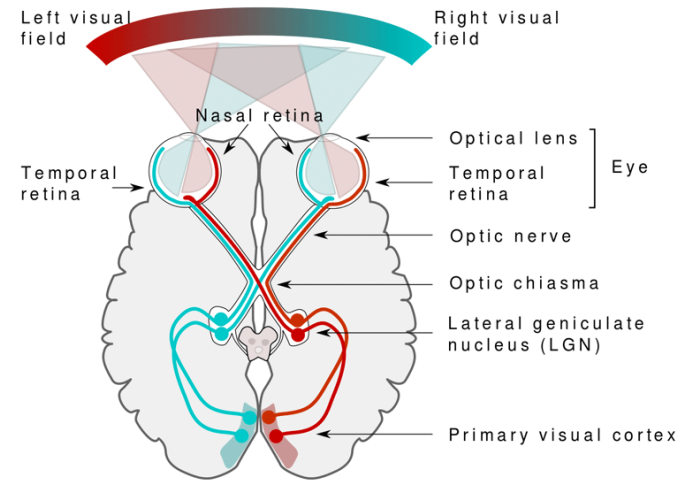
Per capire come ragiona la rete analizziamo quali ingressi eccitano un neurone di ingresso



Perchè Deep Learning?

Perchè tanti strati aiutano?

- **Plausibilità biologica** – per esempio la corteccia visiva
 - Noi già elaboriamo le informazioni visive per strati successivi a partire dalla retina
- **Hastad proof:**
i problemi che possono essere rappresentati con un numero **polinomiale** di nodi in **k strati** potrebbero richiedere un numero **esponenziale** di nodi in **k-1 strati**.



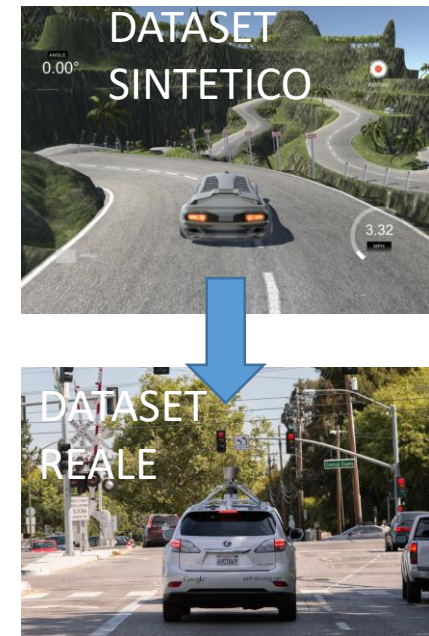
Aggiungere 1 strato può
diminuire il numero di nodi
totali necessari
→ Tanti strati è meglio!

Perchè Deep Learning?

Perchè tanti strati aiutano?

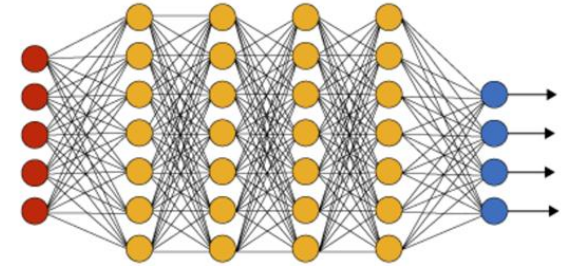
- Le sotto-caratteristiche create in un'architettura profonda possono potenzialmente essere condivise tra più compiti.
- **Trasferimento di conoscenze:** Una rete addestrata su più compiti può essere utilizzata in seguito per compiti simili o correlati.
 - Le conoscenze acquisite durante l'addestramento iniziale possono essere trasferite in modo efficace, riducendo la necessità di addestrare da zero per ogni nuovo compito.

Es: delle feature in un layer che riconoscono un occhio possono essere utili per riconoscere una bocca

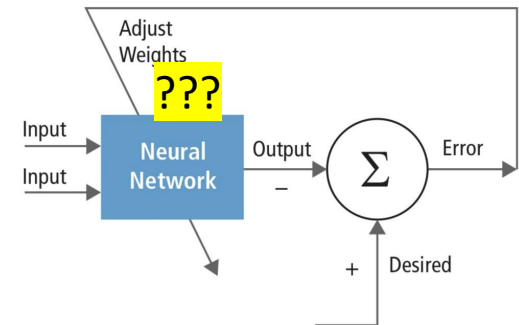


Cosa succede nelle reti molto profonde?

Cosa succede a una rete neurale Mutli-Layer Perceptron (MLP) o similare con molteplici strati nascosti quando viene allenata con il metodo di backpropagation?



- Alcuni anni fa non era possibile per la lentezza della backpropagation e per il fatto che il gradiente retropropagato diventava non usabile



Quali fattori hanno permesso di allenare reti profonde?

- **Hardware**

- L'avvento delle unità di elaborazione grafica (**GPU**) ad alte prestazioni ha reso possibile l'allenamento di reti neurali profonde in tempi ragionevoli. Le GPU sono particolarmente adatte a calcoli intensivi, come quelli richiesti dall'addestramento delle reti neurali.

- **Algoritmi di Ottimizzazione**

- Sono stati sviluppati nuovi algoritmi di ottimizzazione più efficienti, come Adam e RMSprop, che aiutano le reti neurali a convergere più rapidamente.

- **Quantità di Dati**

- Oggi, ci sono molte più fonti di dati disponibili rispetto a 10 anni fa, il che permette l'addestramento su set di dati più ampi, migliorando la capacità predittiva delle reti neurali.

- **Architetture/Modelli Avanzati**

- Sono state introdotte architetture neurali più avanzate, come le reti neurali convoluzionali (**CNN**) per l'elaborazione delle immagini e le reti neurali ricorrenti (**RNN**) per il trattamento delle sequenze, che hanno migliorato le prestazioni in diversi compiti.

NELLA PROSSIMA LEZIONE VEDREMO PROPRIO LE CNN!

L6

Punti principali visti

1. Progettazione di modelli di Machine Learning
 1. Flusso di lavoro
 2. Feature dei modelli da considerare per al scelta
 3. Consigli riassuntivi
2. Introduzione alla Computational Intelligence (CI)
3. Reti neurali feed-forward
4. Curse of Dimensionality
5. Modelli tradizionali di AI (non CI)
 1. Classificatore ai primi vicini kNN
 2. Albero decisionale
6. Caratteristiche dei modelli di Deep Learning